LAPORAN TUGAS TST
BAGIAN B

PEMBANGUNAN API MICROSERVICE KATALOGKU DENGAN POCKETBASE DAN API GATEWAY NGINX PADA SET-TOP BOX (STB) BERBASIS ARMBIAN

Disusun oleh:

Nisrina Zakiyah	18224001
Lukman Hakim Syah Ardhana	18224097

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2026

DAFTAR ISI

DAFTAR ISI **2**
DAFTAR GAMBAR **4**
DAFTAR TABEL **6**
BAGIAN 1: AKSES SERVER MELALUI TAILSCALE **7**
1.1 Pemeriksaan Jaringan Tailscale 7
1.2 Akses SSH ke Server 7
1.3 Pemeriksaan Kondisi Server 8
1.4 Pembuatan Direktori Proyek 9
BAGIAN 2: REMOTE SSH DENGAN VISUAL STUDIO CODE **10**
2.1 Instalasi Ekstensi Remote - SSH 10
2.2 Penambahan Host SSH 10
2.3 Koneksi ke Server 11
2.4 Membuka Folder Proyek 11
BAGIAN 3: MENJALANKAN MICROSERVICE KATALOGKU **13**
3.1 Rancangan Service KatalogKu 13
3.2 Pemindahan Berkas Proyek ke Server 13
3.3 Validasi Konfigurasi Docker Compose 14
3.4 Build dan Menjalankan Container 15
3.5 Pembuatan Akun Superuser 16
BAGIAN 4: KONFIGURASI KOLEKSI POCKETBASE **17**
4.1 Login Dashboard Product Service 17
4.2 Login Dashboard Account Service 17
4.3 Import Koleksi products 18
4.4 Import Koleksi users dan favorites 20
BAGIAN 5: PENGISIAN DATA PRODUK **23**
5.1 Instalasi REST Client 23
5.2 Penyesuaian Berkas seed.http 23
5.3 Login Superuser 23
5.4 Penambahan Data Produk 24
BAGIAN 6: PENGUJIAN API DENGAN REST CLIENT **30**
6.1 Penyesuaian Berkas api.http 30
6.2 Cek Gateway 30
6.3 Daftar Produk 30
6.4 Pencarian Produk dengan Filter 31
6.5 Detail Produk 31
6.6 Pendaftaran Akun 32
6.7 Login 33
6.8 Tambah Favorit 34
6.9 Lihat Favorit Saya 34
6.10 Hapus Favorit 35
6.11 Ringkasan Pengujian REST Client 35
BAGIAN 7: PENGUJIAN API DENGAN POSTMAN **37**

7.1 Instalasi Postman 37
7.2 Import Koleksi dan Environment 37
7.3 Pengaturan Environment ServerLAN 38
7.4 Hasil Pengujian Postman 39
BAGIAN 8: BACKUP DATA DAN UJI REBOOT 42
8.1 Backup Product Service 42
8.2 Backup Account Service 43
8.3 Pemeriksaan Container dan Restart Policy 43
8.4 Bukti Hasil: Uji Reboot STB 44

DAFTAR GAMBAR

Gambar 1.1 Hasil perintah tailscale status 7
Gambar 1.2 Login SSH sebagai root ke 100.79.171.72 8
Gambar 1.3 Hasil pemeriksaan sistem, penyimpanan, dan Docker pada server 9
Gambar 1.4 Pembuatan direktori proyek katalogku-pocketbase 9
Gambar 2.1 Ekstensi Remote - SSH pada VS Code 10
Gambar 2.2 Pengisian perintah koneksi SSH 10
Gambar 2.3 Pemilihan berkas konfigurasi SSH 10
Gambar 2.4 Isi berkas ~/.ssh/config 11
Gambar 2.5 VS Code terhubung ke server melalui Remote - SSH 11
Gambar 2.6 Folder katalogku-pocketbase terbuka pada Explorer VS Code 11
Gambar 2.7 Hasil pwd dan ls -lah pada direktori proyek 12
Gambar 3.1 Penyalinan berkas proyek ke server dengan scp 13
Gambar 3.2 Struktur berkas proyek pada server 14
Gambar 3.3 Hasil docker compose config 15
Gambar 3.4 Hasil docker compose up -d --build 16
Gambar 3.5 Hasil docker compose ps 16
Gambar 3.6 Superuser product-service berhasil dibuat 16
Gambar 3.7 Superuser account-service berhasil dibuat 16
Gambar 4.1 Halaman login dashboard product-service 17
Gambar 4.2 Dashboard product-service setelah login 17
Gambar 4.3 Halaman login dashboard account-service 18
Gambar 4.4 Dashboard account-service setelah login 18
Gambar 4.5 Halaman Import collections pada dashboard 19
Gambar 4.6 Pemilihan berkas product_collections.json 19
Gambar 4.7 Preview perubahan sebelum import koleksi products 20
Gambar 4.8 Koleksi products berhasil ditambahkan 20
Gambar 4.9 Pemilihan berkas account_collections.json 21
Gambar 4.10 Preview perubahan sebelum import koleksi account-service 21
Gambar 4.11 Koleksi users dan favorites berhasil ditambahkan 22
Gambar 5.1 Ekstensi REST Client pada VS Code 23
Gambar 5.2 Request 0. Login superuser pada seed.http 24
Gambar 5.3 Respons login superuser 24
Gambar 5.4 Request 1. Kemeja Batik Parang 25
Gambar 5.5 Respons penambahan produk Kemeja Batik Parang 25
Gambar 5.6 Respons penambahan produk Kopi Gayo 250 gr 26
Gambar 5.7 Respons penambahan produk Teh Hijau Organik 27
Gambar 5.8 Respons penambahan produk Earphone Bluetooth 28
Gambar 5.9 Respons penambahan produk Power Bank 10000 mAh 29
Gambar 6.1 Request 1. Cek gateway 30
Gambar 6.2 Respons cek gateway 30
Gambar 6.3 Respons daftar produk (5 terbaru) 31
Gambar 6.4 Respons pencarian produk dengan filter 31

Gambar 6.5 Respons detail produk 32
Gambar 6.6 Respons pendaftaran akun 33
Gambar 6.7 Respons login 33
Gambar 6.8 Respons tambah favorit 34
Gambar 6.9 Respons lihat favorit 35
Gambar 6.10 Respons hapus favorit 35
Gambar 7.1 Halaman unduhan Postman 37
Gambar 7.2 Jendela Import pada Postman 37
Gambar 7.3 Pemilihan berkas koleksi dan environment Postman 38
Gambar 7.4 Konfirmasi import koleksi dan environment 38
Gambar 7.5 Environment KatalogKu - ServerLAN 38
Gambar 7.6 Variabel baseUrl sebelum diubah 39
Gambar 7.7 Postman: cek gateway 39
Gambar 7.8 Postman: daftar produk 39
Gambar 7.9 Postman: cari produk 39
Gambar 7.10 Postman: detail produk 40
Gambar 7.11 Postman: daftar akun 40
Gambar 7.12 Postman: login 40
Gambar 7.13 Postman: tambah favorit 41
Gambar 7.14 Postman: lihat favorit 41
Gambar 8.1 Isi koleksi products pada dashboard product-service 42
Gambar 8.2 Halaman Backups pada dashboard product-service 42
Gambar 8.3 Backup product-service berhasil dibuat 43
Gambar 8.4 Backup account-service berhasil dibuat 43
Gambar 8.5 Daftar container yang berjalan 43
Gambar 8.6 Restart policy seluruh container 44
Gambar 8.7 Container aktif kembali setelah STB reboot 44

DAFTAR TABEL

Tabel 3.1 Rancangan service KatalogKu.....13
Tabel 5.1 Data produk hasil seed.....29
Tabel 6.1 Ringkasan hasil pengujian API dengan REST Client.....35

BAGIAN 1: AKSES SERVER MELALUI TAILSCALE

1.1 Pemeriksaan Jaringan Tailscale

Server yang digunakan pada Bagian B masih sama dengan Bagian A, yaitu set-top box (STB) berbasis Armbian yang sudah terpasang Docker, MySQL, dan phpMyAdmin. Pada bagian ini server diakses dari MacBook melalui jaringan Tailscale, sehingga STB dapat dijangkau dengan alamat IP Tailscale tanpa harus terhubung langsung menggunakan kabel Ethernet. Daftar perangkat di dalam jaringan Tailscale diperiksa dengan perintah berikut.

```
tailscale status
```

Gambar 1.1 memperlihatkan bahwa STB dengan hostname armbian (sistem operasi linux) terdaftar pada alamat 100.79.171.72. Alamat tersebut digunakan sebagai IP server pada seluruh langkah berikutnya.

```
bellechillguy@Nisrinas-MacBook-Air ~ % tailscale status
100.78.171.14  nisrinas-macbook-air-1  nisrinazakiyah1820@  macOS  -
100.79.171.72  armbian                 nisrinazakiyah1820@  linux  offline, last seen 10d ago
100.95.248.5   nisrinas-macbook-air    nisrinazakiyah1820@  macOS  offline, last seen 10d ago
```

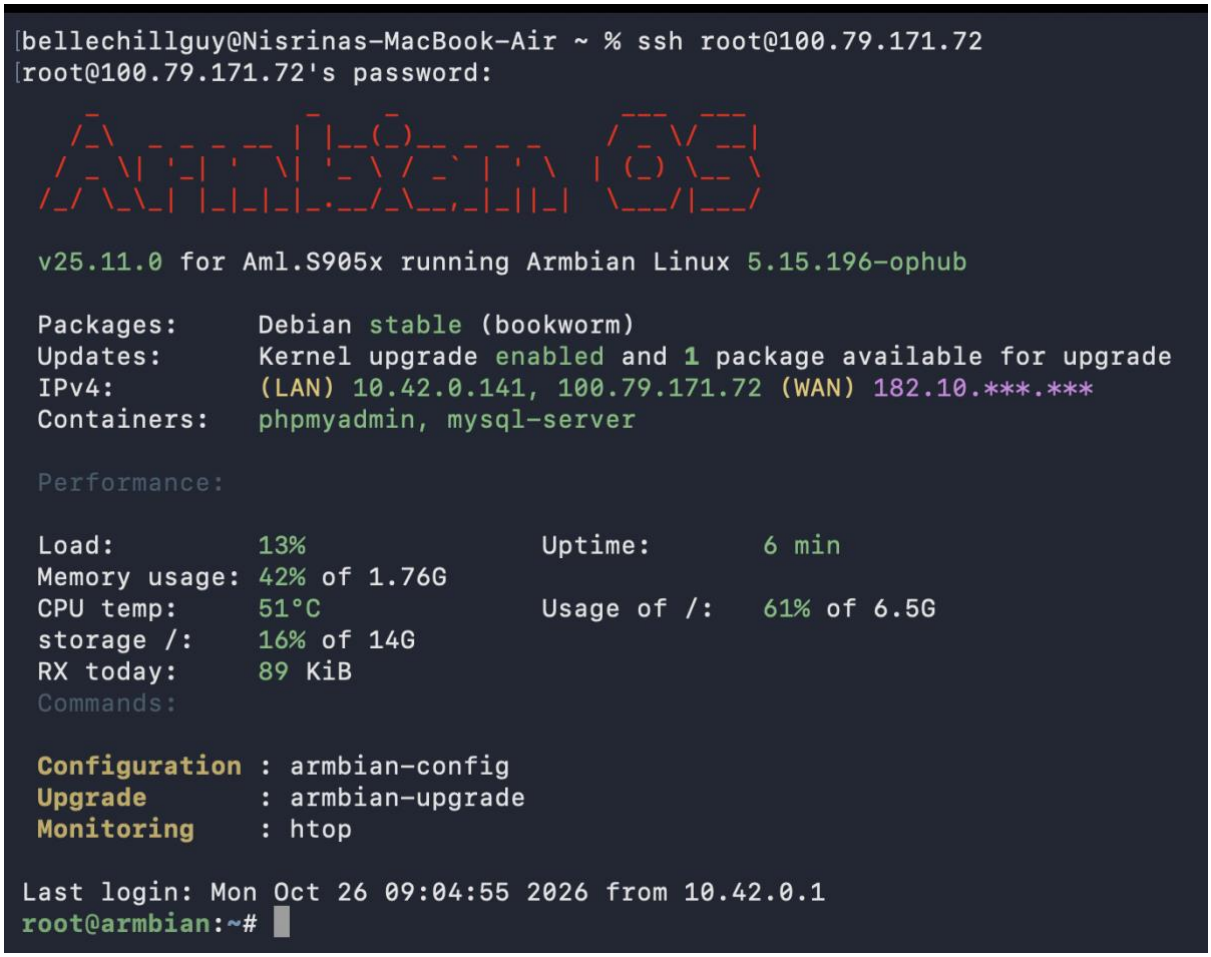
Gambar 1.1 Hasil perintah tailscale status

1.2 Akses SSH ke Server

Server diakses dari terminal MacBook melalui SSH sebagai pengguna root dengan alamat IP Tailscale.

```
ssh root@100.79.171.72
user: root
password: *******
```

Gambar 1.2 menampilkan permintaan kata sandi (root@100.79.171.72's password), kemudian banner Armbian setelah login berhasil. Pada banner terlihat alamat IPv4 server, yaitu 10.42.0.141 (LAN) dan 100.79.171.72, serta container phpmyadmin dan mysql-server dari Bagian A yang sudah berjalan.



Gambar 1.2 Login SSH sebagai root ke 100.79.171.72

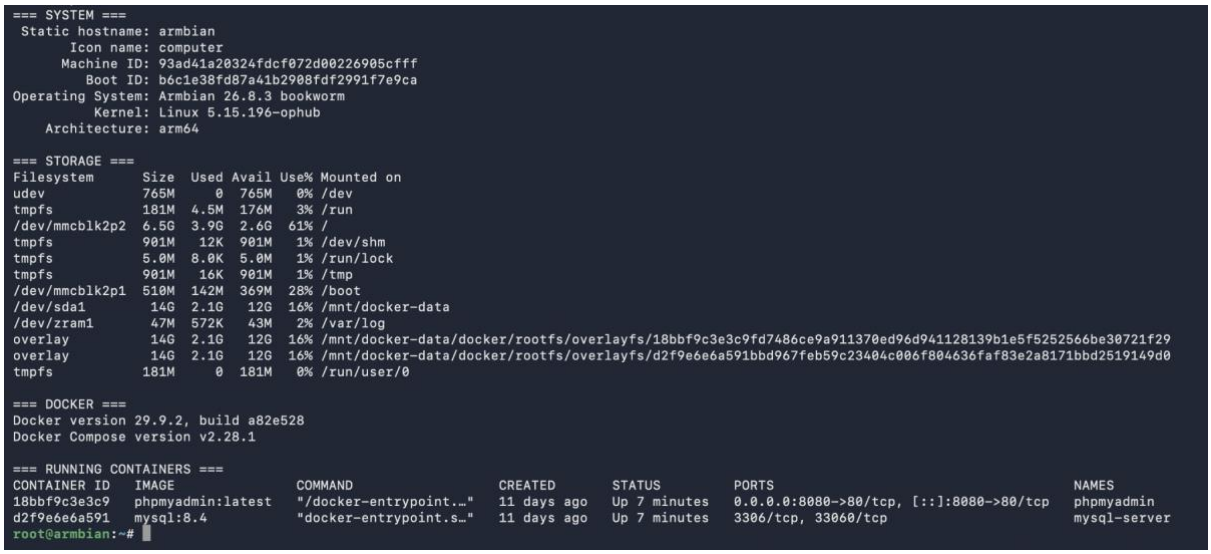
1.3 Pemeriksaan Kondisi Server

Sebelum membangun microservice, kondisi sistem, penyimpanan, versi Docker, dan container yang sedang berjalan diperiksa sekaligus dengan rangkaian perintah berikut.

```
echo "=== SYSTEM ==="
hostnamectl
echo
echo "=== STORAGE ==="
df -h
echo
echo "=== DOCKER ==="
docker --version
docker compose version
echo
echo "=== RUNNING CONTAINERS ==="
docker ps
```

Hasilnya (Gambar 1.3) menunjukkan server menjalankan Armbian 26.8.3 bookworm dengan kernel Linux 5.15.196-ophub dan arsitektur arm64. Flashdisk /dev/sda1 berukuran 14 GB sudah ter-mount pada /mnt/docker-data, Docker versi 29.9.2 dan Docker Compose v2.28.1 sudah

terpasang, serta container phpmyadmin (port 8080) dan mysql-server dari Bagian A sedang berjalan.



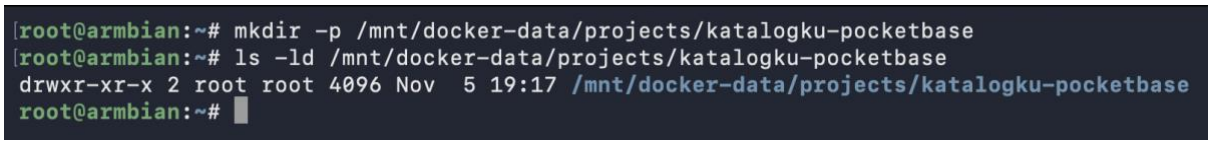
Gambar 1.3 Hasil pemeriksaan sistem, penyimpanan, dan Docker pada server

1.4 Pembuatan Direktori Proyek

Direktori proyek katalogku-pocketbase dibuat di dalam flashdisk (/mnt/docker-data/projects) agar seluruh berkas proyek tersimpan pada penyimpanan yang sama dengan data Docker.

```
mkdir -p /mnt/docker-data/projects/katalogku-pocketbase
ls -ld /mnt/docker-data/projects/katalogku-pocketbase
```

Gambar 1.4 menunjukkan direktori berhasil dibuat dengan pemilik root.

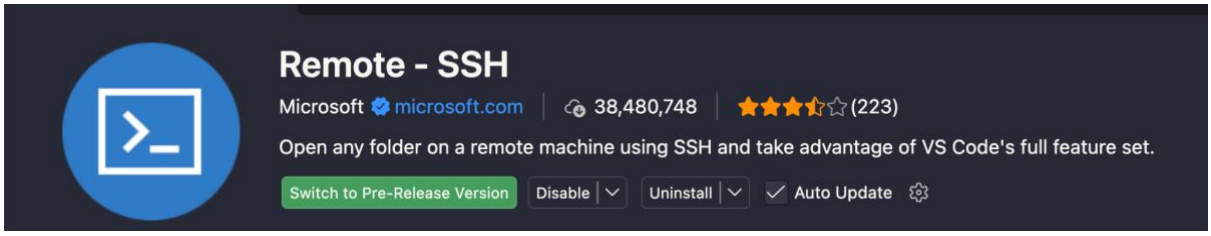


Gambar 1.4 Pembuatan direktori proyek katalogku-pocketbase

BAGIAN 2: REMOTE SSH DENGAN VISUAL STUDIO CODE

2.1 Instalasi Ekstensi Remote - SSH

Agar berkas di server dapat dibuka dan diedit langsung dari VS Code, dipasang ekstensi Remote - SSH dari Microsoft melalui menu Extensions.

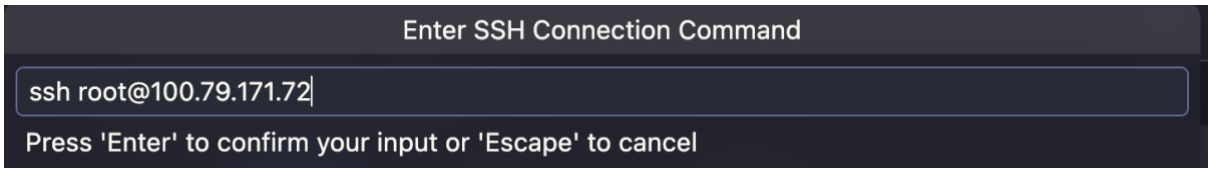


Gambar 2.1 Ekstensi Remote - SSH pada VS Code

2.2 Penambahan Host SSH

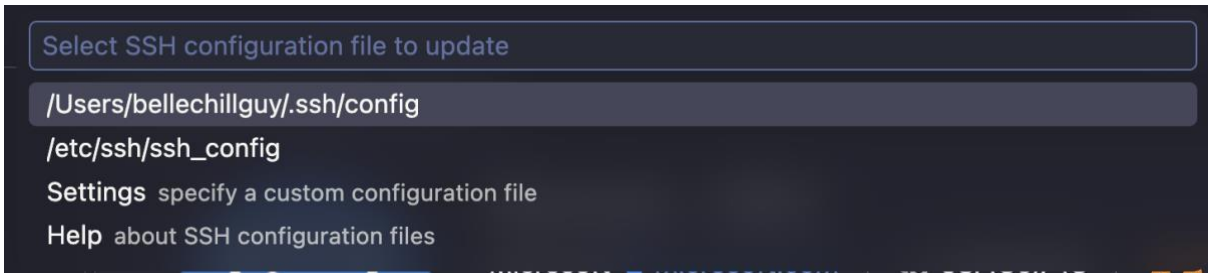
Pada VS Code ditekan Cmd + Shift + P, lalu dipilih Remote-SSH: Add New SSH Host. Perintah SSH berikut dimasukkan ke kolom Enter SSH Connection Command.

```
ssh root@100.79.171.72
```



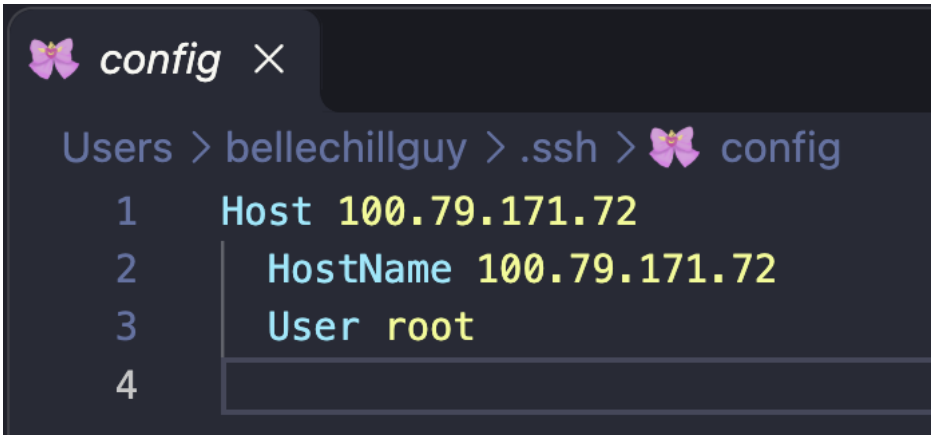
Gambar 2.2 Pengisian perintah koneksi SSH

Selanjutnya dipilih berkas konfigurasi /Users/bellechillguy/.ssh/config sebagai tempat penyimpanan host baru.



Gambar 2.3 Pemilihan berkas konfigurasi SSH

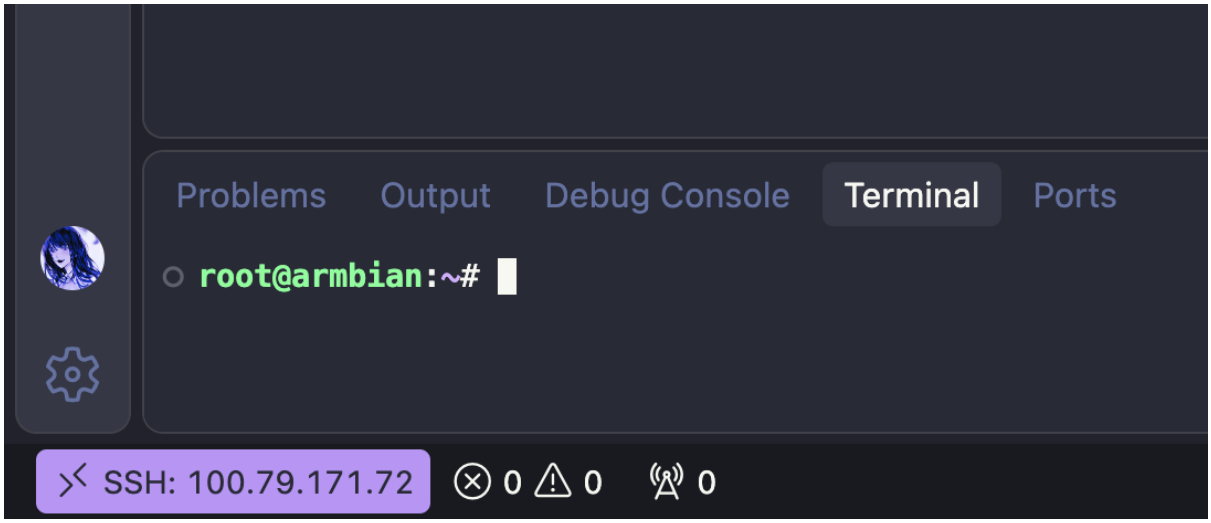
Gambar 2.4 memperlihatkan isi berkas config yang terbentuk, yaitu Host 100.79.171.72 dengan HostName 100.79.171.72 dan User root.



Gambar 2.4 Isi berkas ~/.ssh/config

2.3 Koneksi ke Server

Koneksi dibuka dengan menekan Cmd + Shift + P, memilih Remote-SSH: Connect to Host..., lalu memilih root@100.79.171.72 dan sistem operasi Linux. Setelah terhubung, status bar VS Code menampilkan SSH: 100.79.171.72 dan terminal terintegrasi langsung berada pada prompt root@armbian (Gambar 2.5).

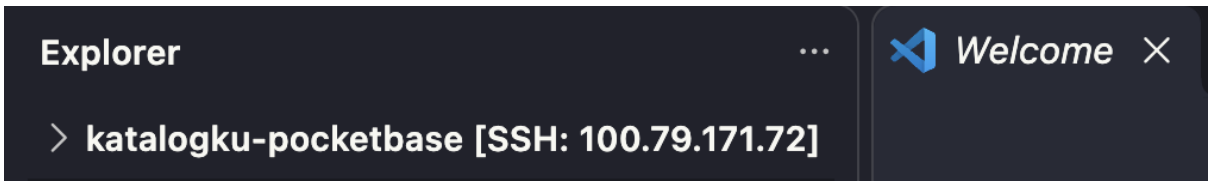


Gambar 2.5 VS Code terhubung ke server melalui Remote - SSH

2.4 Membuka Folder Proyek

Folder proyek dibuka melalui menu File, Open Folder, dengan path berikut.

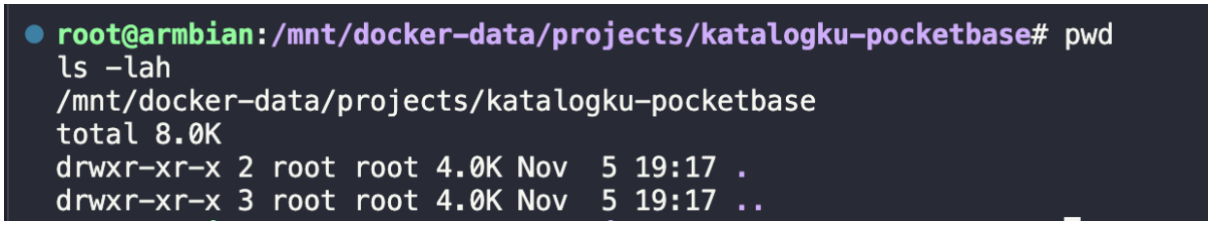
```
/mnt/docker-data/projects/katalogku-pocketbase
```



Gambar 2.6 Folder katalogku-pocketbase terbuka pada Explorer VS Code

Posisi direktori dan isinya diperiksa dari terminal VS Code. Pada tahap ini direktori masih kosong (Gambar 2.7).

```
pwd
ls -lah
```



Gambar 2.7 Hasil pwd dan ls -lah pada direktori proyek

BAGIAN 3: MENJALANKAN MICROSERVICE KATALOGKU

3.1 Rancangan Service KatalogKu

Microservice KatalogKu dibangun tanpa coding menggunakan PocketBase, yaitu backend open source (lisensi MIT) yang menyediakan database SQLite, dashboard admin, REST API otomatis, serta fitur login dan API Rules. Dua instance PocketBase digabungkan dengan API Gateway Nginx di dalam Docker Compose dengan rancangan sebagai berikut.

Tabel 3.1 Rancangan service KatalogKu

Service	Isi	Port	Prefix URL (gateway)
gateway (Nginx)	Satu pintu masuk yang meneruskan request ke service yang tepat	8081	/api/...
product-service	PocketBase #1: koleksi products	8091 (dashboard)	/api/products
account-service	PocketBase #2: koleksi users (login) dan favorites	8092 (dashboard)	/api/auth, /api/favorites

Setiap PocketBase memiliki database SQLite sendiri (database-per-service), sehingga tidak ada tabel yang dipakai bersama. Koleksi favorites hanya menyimpan product_id, sedangkan detail produk tetap diambil dari product-service. Karena port 8080 pada server sudah digunakan oleh phpMyAdmin dari Bagian A, gateway dipublikasikan pada port 8081.

3.2 Pemindahan Berkas Proyek ke Server

Berkas proyek katalogku-pocketbase yang sudah diunduh ke MacBook dipindahkan ke direktori proyek di server menggunakan scp dari terminal MacBook.

```
scp -r /Users/bellechillguy/Downloads/katalogku-pocketbase/* \
  root@100.79.171.72:/mnt/docker-data/projects/katalogku-pocketbase/
```

Gambar 3.1 menunjukkan seluruh berkas, antara lain api.http, docker-compose.yml, nginx.conf, Dockerfile, berkas skema koleksi, berkas Postman, README.md, dan seed.http, berhasil tersalin 100%.



Gambar 3.1 Penyalinan berkas proyek ke server dengan scp

Struktur berkas di server kemudian diperiksa dari terminal SSH.

```
cd /mnt/docker-data/projects/katalogku-pocketbase
find . -maxdepth 3 -type f | sort
```

Gambar 3.2 memperlihatkan struktur proyek yang sesuai panduan: docker-compose.yml, gateway/nginx.conf, pocketbase/Dockerfile, pb_schema/ (product_collections.json dan account_collections.json), postman/ (koleksi dan tiga environment), api.http, seed.http, dan README.md.



Gambar 3.2 Struktur berkas proyek pada server

3.3 Validasi Konfigurasi Docker Compose

Konfigurasi docker-compose.yml divalidasi terlebih dahulu sebelum container dijalankan.

```
docker compose config
```

Hasil pada Gambar 3.3 menunjukkan tiga service, yaitu account-service (8092 ke 8090), gateway dengan image nginx:1.28-alpine (8081 ke 80), dan product-service (8091 ke 8090). Kedua PocketBase dibangun dari Dockerfile yang sama, menggunakan restart policy unless-stopped, dan menyimpan data pada volume terpisah (account_data dan product_data). Peringatan TUNNEL_TOKEN muncul karena token Cloudflare baru digunakan pada Bagian C.



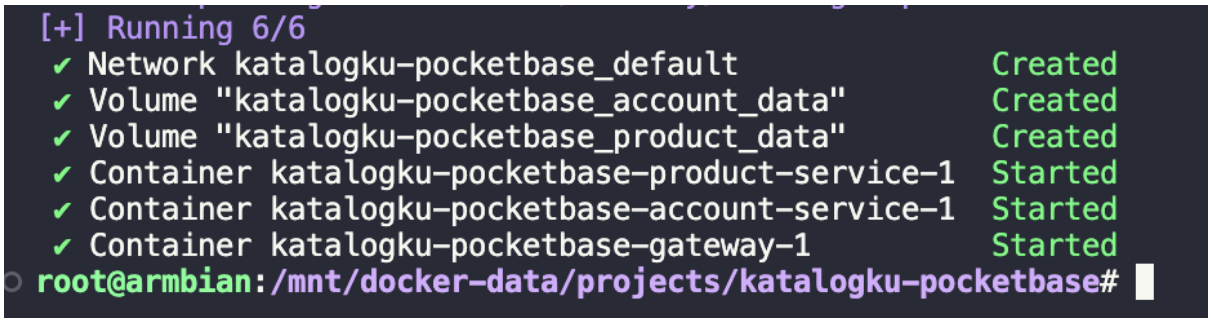
Gambar 3.3 Hasil docker compose config

3.4 Build dan Menjalankan Container

Image PocketBase dibangun dan seluruh container dijalankan di latar belakang.

```
docker compose up -d --build
```

Gambar 3.4 menunjukkan network katalogku-pocketbase_default dan dua volume berhasil dibuat, serta container product-service, account-service, dan gateway berstatus Started.



Gambar 3.4 Hasil docker compose up -d --build

Status container kemudian diperiksa.

```
docker compose ps
```

Ketiga container berstatus Up dengan pemetaan port 8081 (gateway), 8091 (product-service), dan 8092 (account-service), sebagaimana terlihat pada Gambar 3.5.

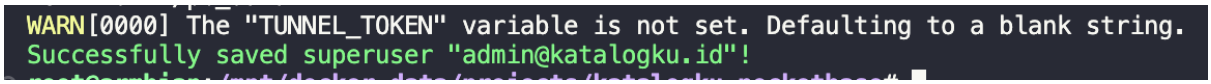


Gambar 3.5 Hasil docker compose ps

3.5 Pembuatan Akun Superuser

Akun admin (superuser) dashboard dibuat pada kedua service PocketBase. Password yang digunakan bukan password contoh dari panduan.

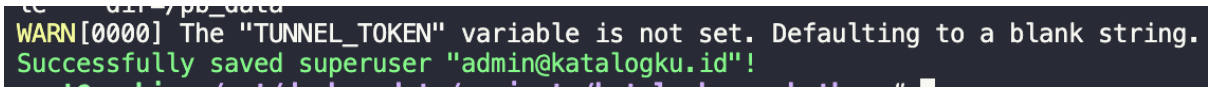
```
docker compose exec product-service pocketbase superuser upsert \
  admin@katalogku.id '*******' --dir=/pb_data
```



Gambar 3.6 Superuser product-service berhasil dibuat

```
docker compose exec account-service pocketbase superuser upsert \
  admin@katalogku.id '*******' --dir=/pb_data
```

Kedua perintah menghasilkan pesan Successfully saved superuser "admin@katalogku.id" (Gambar 3.6 dan Gambar 3.7).



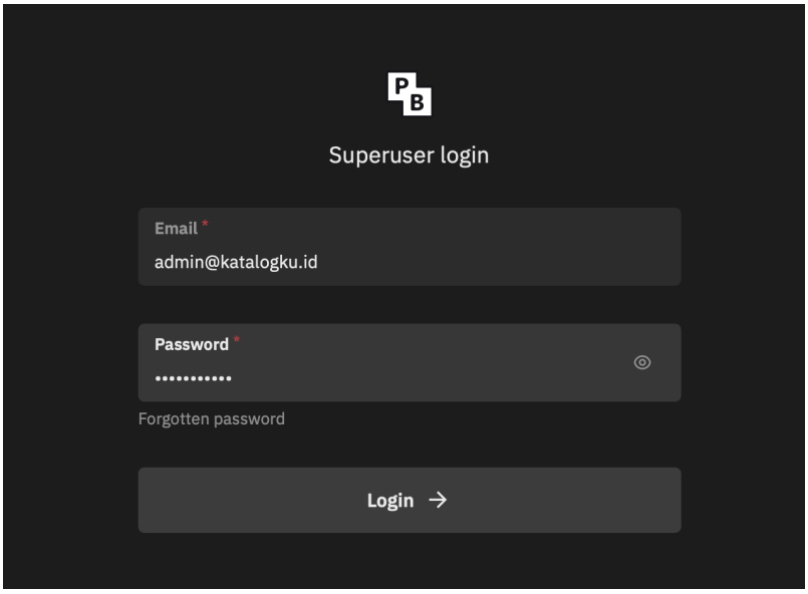
Gambar 3.7 Superuser account-service berhasil dibuat

BAGIAN 4: KONFIGURASI KOLEKSI POCKETBASE

4.1 Login Dashboard Product Service

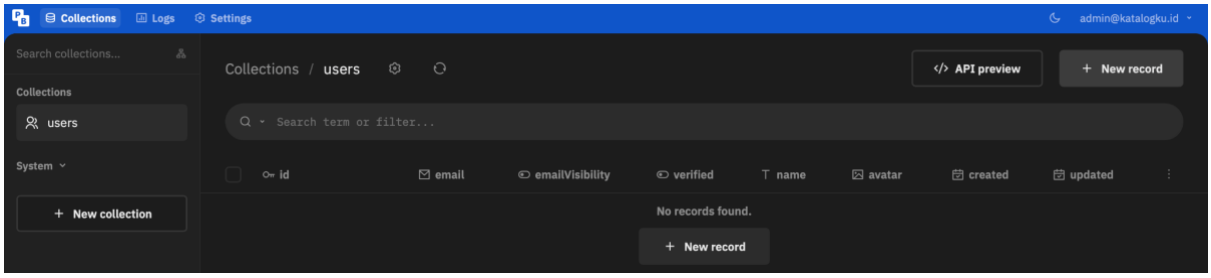
Dashboard product-service dibuka melalui peramban dengan kredensial superuser berikut.

```
http://100.79.171.72:8091/_/#/login
email: admin@katalogku.id
password: *******
```



Gambar 4.1 Halaman login dashboard product-service

Setelah login berhasil, dashboard menampilkan halaman Collections dengan koleksi bawaan users yang masih kosong (Gambar 4.2).

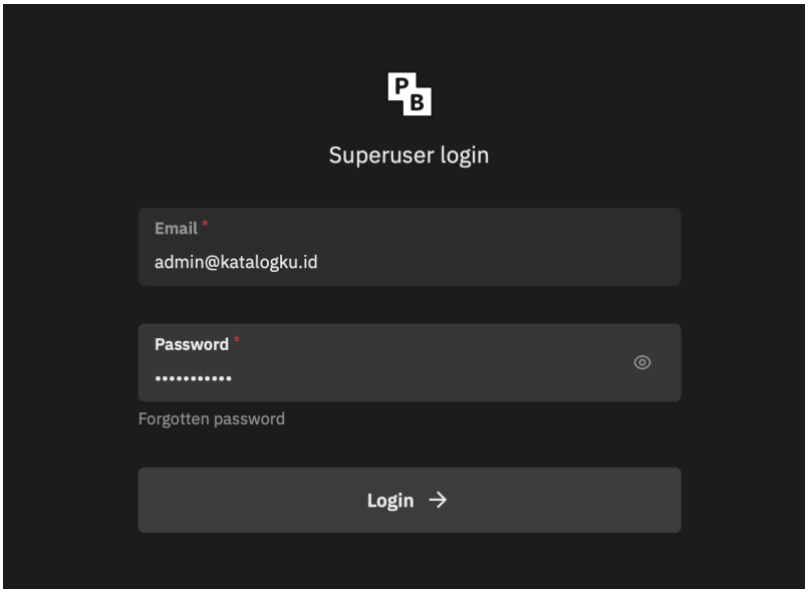


Gambar 4.2 Dashboard product-service setelah login

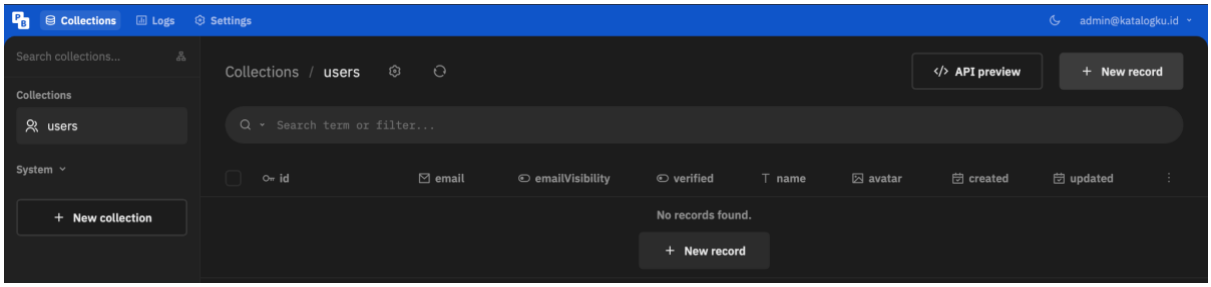
4.2 Login Dashboard Account Service

Dengan cara yang sama, dashboard account-service dibuka pada port 8092.

```
http://100.79.171.72:8092/_/#/login
email: admin@katalogku.id
password: *******
```



Gambar 4.3 Halaman login dashboard account-service

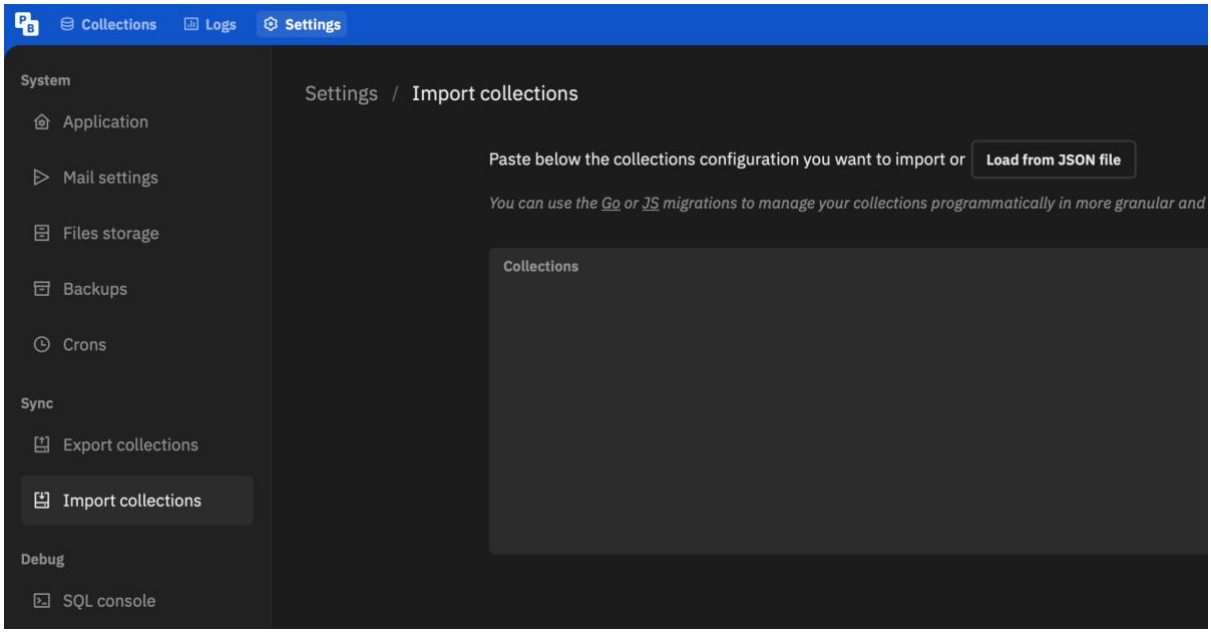


Gambar 4.4 Dashboard account-service setelah login

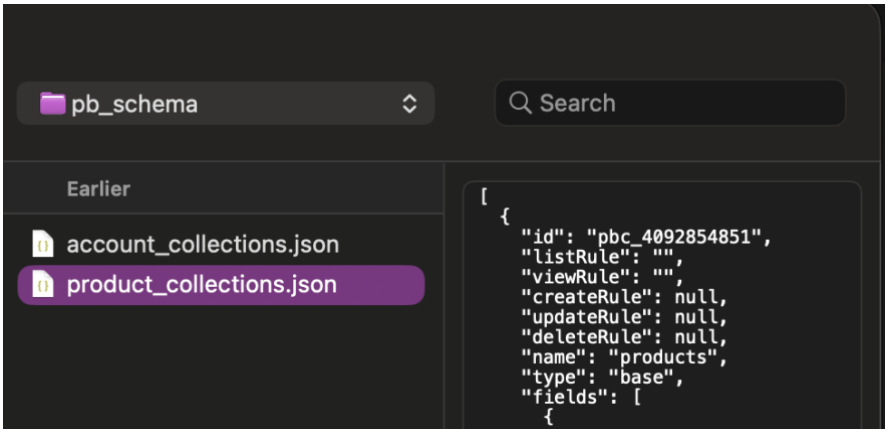
4.3 Import Koleksi products

Koleksi products dibuat dengan cara cepat, yaitu import skema yang sudah disediakan. Langkah yang dilakukan pada dashboard product-service (http://100.79.171.72:8091/_/) adalah sebagai berikut.

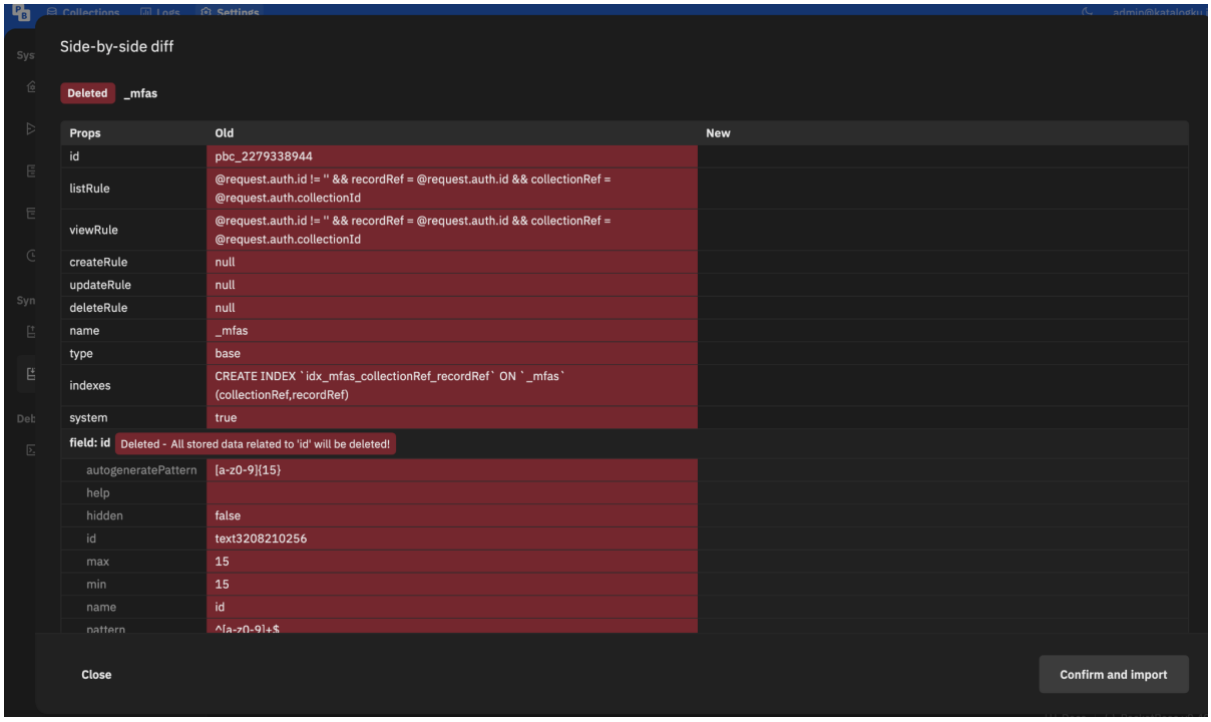
1. Membuka menu Collections.
2. Membuka Settings, lalu memilih Import collections (Gambar 4.5).
3. Memilih opsi Load from JSON file.
4. Membuka folder proyek lokal di MacBook, yaitu Downloads/katalogku-pocketbase/pb_schema/.
5. Memilih berkas product_collections.json (Gambar 4.6).
6. Memeriksa preview perubahan, lalu menekan Confirm and import (Gambar 4.7).



Gambar 4.5 Halaman Import collections pada dashboard

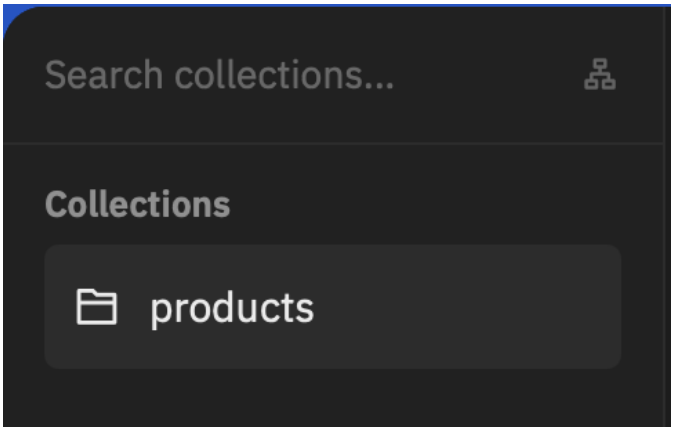


Gambar 4.6 Pemilihan berkas product_collections.json



Gambar 4.7 Preview perubahan sebelum import koleksi products

Setelah import dikonfirmasi, koleksi products muncul pada daftar Collections (Gambar 4.8). Sesuai skema, koleksi ini memiliki field title, description, category, price, rating, stock, dan thumbnail. API Rules List/View dibuka untuk publik, sedangkan Create/Update/Delete tetap terkunci sehingga hanya superuser yang dapat mengubah produk.



Gambar 4.8 Koleksi products berhasil ditambahkan

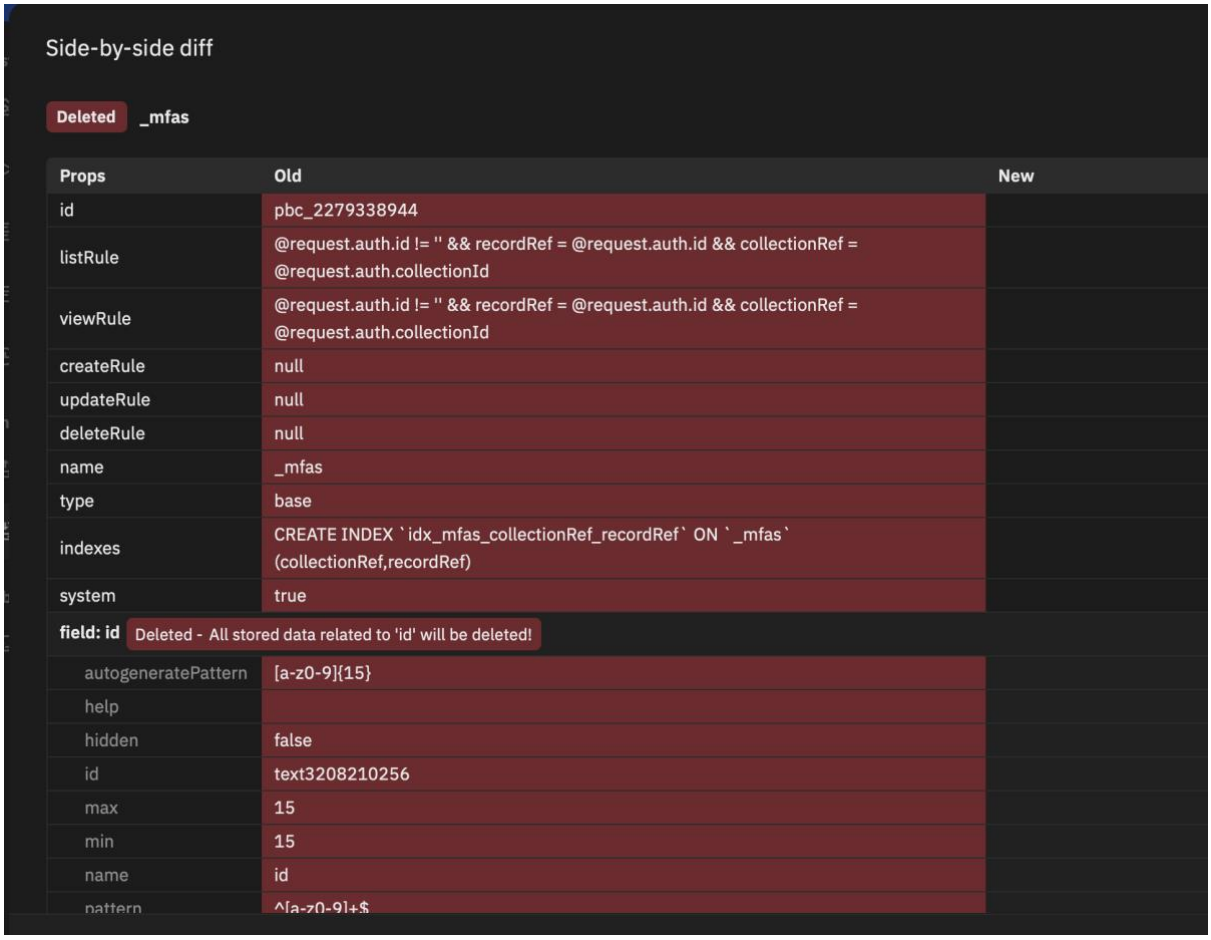
4.4 Import Koleksi users dan favorites

Langkah yang sama dilakukan pada dashboard account-service (http://100.79.171.72:8092/_/), yaitu membuka Collections, lalu Settings dan Import collections seperti pada Gambar 4.5. Berkas yang dipilih adalah account_collections.json dari folder Downloads/katalogku-pocketbase/pb_schema/.



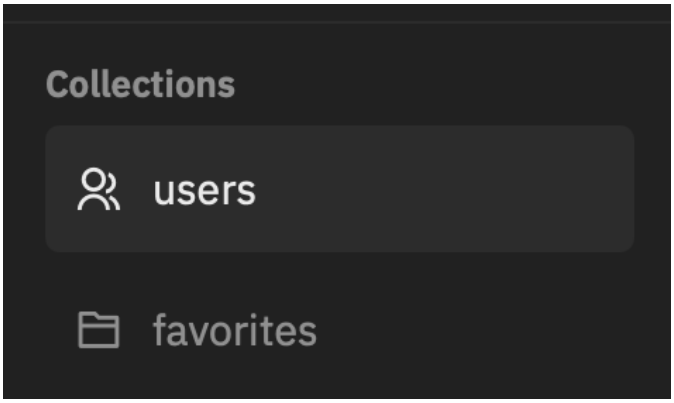
Gambar 4.9 Pemilihan berkas account_collections.json

Preview perubahan diperiksa, lalu import dikonfirmasi.



Gambar 4.10 Preview perubahan sebelum import koleksi account-service

Gambar 4.11 menunjukkan dua koleksi pada account-service, yaitu users (koleksi tipe Auth untuk login) dan favorites. API Rules pada favorites membatasi agar setiap user hanya dapat melihat, menambah, dan menghapus favorit miliknya sendiri (user = @request.auth.id).

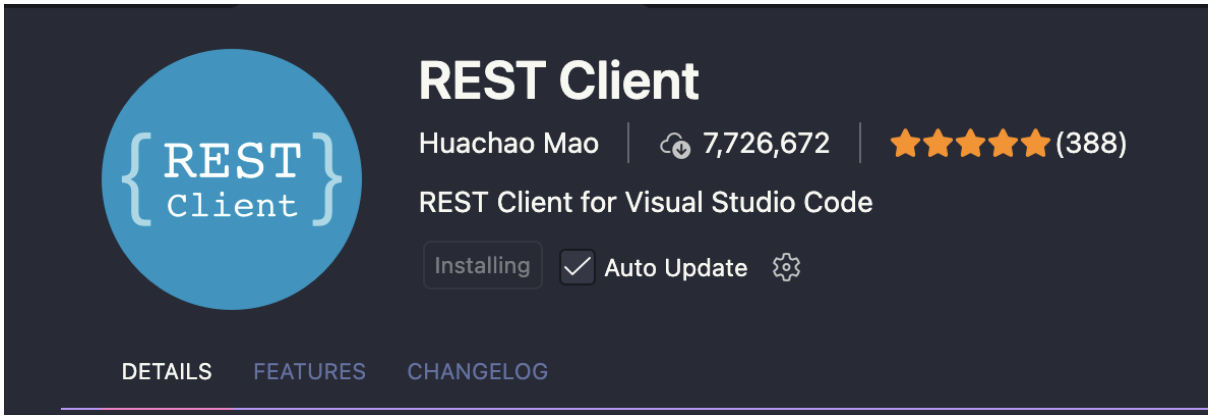


Gambar 4.11 Koleksi users dan favorites berhasil ditambahkan

BAGIAN 5: PENGISIAN DATA PRODUK

5.1 Instalasi REST Client

Data produk diisi menggunakan berkas seed.http yang dijalankan dengan ekstensi REST Client (Huachao Mao) pada VS Code.



Gambar 5.1 Ekstensi REST Client pada VS Code

5.2 Penyesuaian Berkas seed.http

Pada VS Code Remote SSH, berkas seed.http dibuka dan alamat PocketBase diubah dari localhost menjadi IP server.

```
@pbUrl = http://localhost:8091
```

menjadi:

```
@pbUrl = http://100.79.171.72:8091
```

Password contoh pada request login superuser juga diganti dengan password superuser product-service yang dibuat pada subbab 3.5.

```
"identity": "admin@katalogku.id",
"password": "*******"
```

5.3 Login Superuser

Request 0. Login superuser dijalankan dengan menekan tulisan Send Request di atas request tersebut.

```
### 0. Login superuser
# @name su
Send Request
POST {{pbUrl}}/api/collections/_superusers/auth-with-password
Content-Type: application/json
```

Gambar 5.2 Request 0. Login superuser pada seed.http

Respons HTTP/1.1 200 OK pada Gambar 5.3 berisi record superuser admin@katalogku.id beserta token. Token ini otomatis disimpan sebagai variabel suToken dan dipakai sebagai header Authorization pada request penambahan produk.

Response(489ms)


```
HTTP/1.1 200 OK
Content-Type: application/json
Cross-Origin-Opener-Policy: same-origin
Vary: Origin
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
X-Xss-Protection: 1; mode=block
Date: Thu, 05 Nov 2026 14:10:06 GMT
Content-Length: 475
Connection: close

{
  "record": {
    "collectionId": "pbc_3142635823",
    "collectionName": "_superusers",
    "created": "2026-11-05 13:45:23.144Z",
    "email": "admin@katalogku.id",
    "emailVisibility": false,
    "id": "nzr98gmmth90asv",
    "updated": "2026-11-05 13:45:23.144Z",
    "verified": true
  },
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJjb2xsZWN0aW9uSWQiOiJwYmNfMzE0MjYzNTgyMyIsImV4cCI6MTc5Mzk3NDIwNiwiaWQiOiJuenI5OGdtbXRoOTBhc3YiLCJyZWZyZXNoYWJsZSI6dHJ1ZSwidHlwZSI6ImF1dGgifQ.WIMOVrpuPPDbzSNdt7Ks_pnEph6u78x3mzwiPmRjkiQ"
}
```


Gambar 5.3 Respons login superuser

5.4 Penambahan Data Produk

Request 1. Kemeja Batik Parang dijalankan dengan menekan Send Request.

```
### 1. Kemeja Batik Parang
Send Request
POST {{pbUrl}}/api/collections/products/records
Authorization: {{suToken}}
Content-Type: application/json

{
  "title": "Kemeja Batik Parang",
  "description": "Kemeja batik motif parang, bahan katun.",
  "category": "fashion",
  "price": 25,
  "rating": 4.6,
  "stock": 40,
  "thumbnail": "https://picsum.photos/id/21/300"
}
```

Gambar 5.4 Request 1. Kemeja Batik Parang

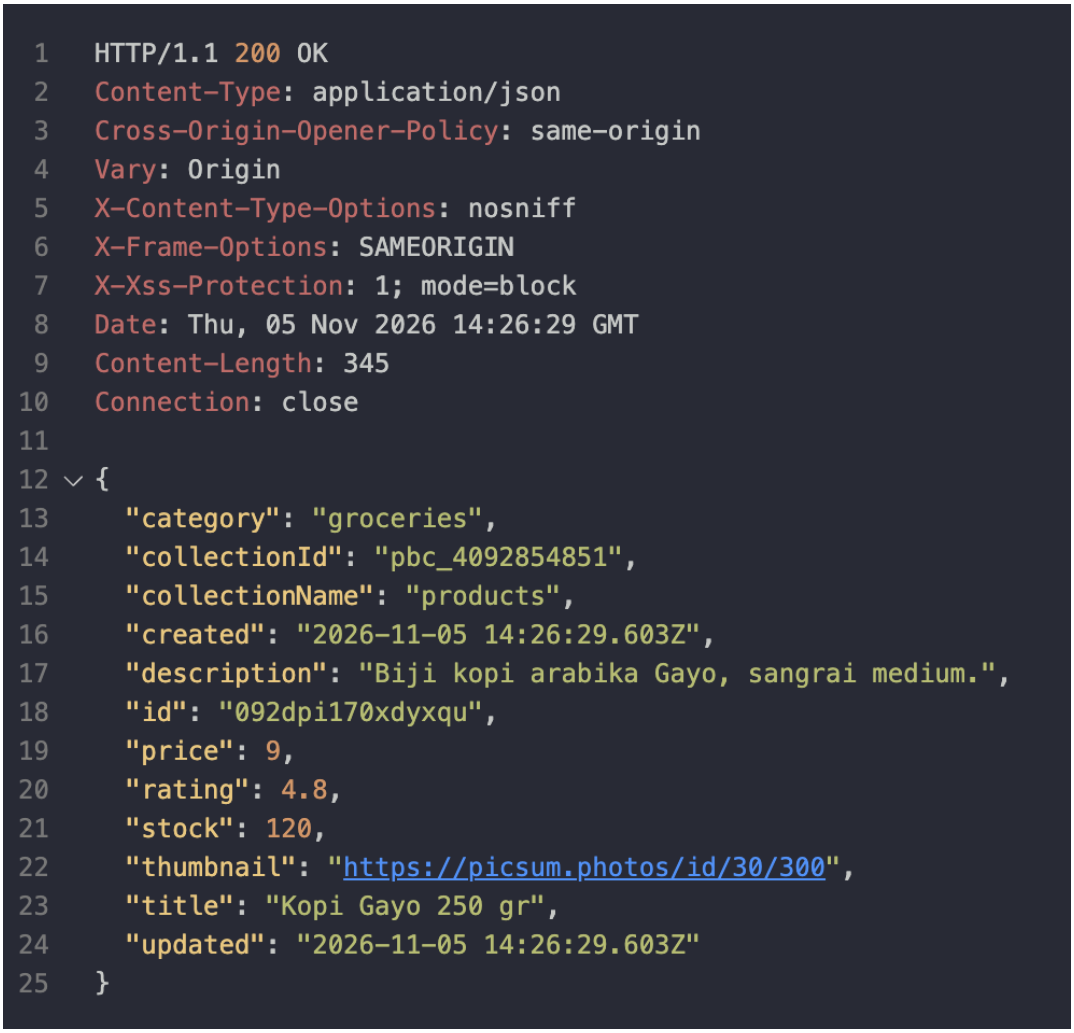
Response(37ms)

```
HTTP/1.1 200 OK
Content-Type: application/json
Cross-Origin-Opener-Policy: same-origin
Vary: Origin
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
X-Xss-Protection: 1; mode=block
Date: Thu, 05 Nov 2026 14:23:17 GMT
Content-Length: 346
Connection: close

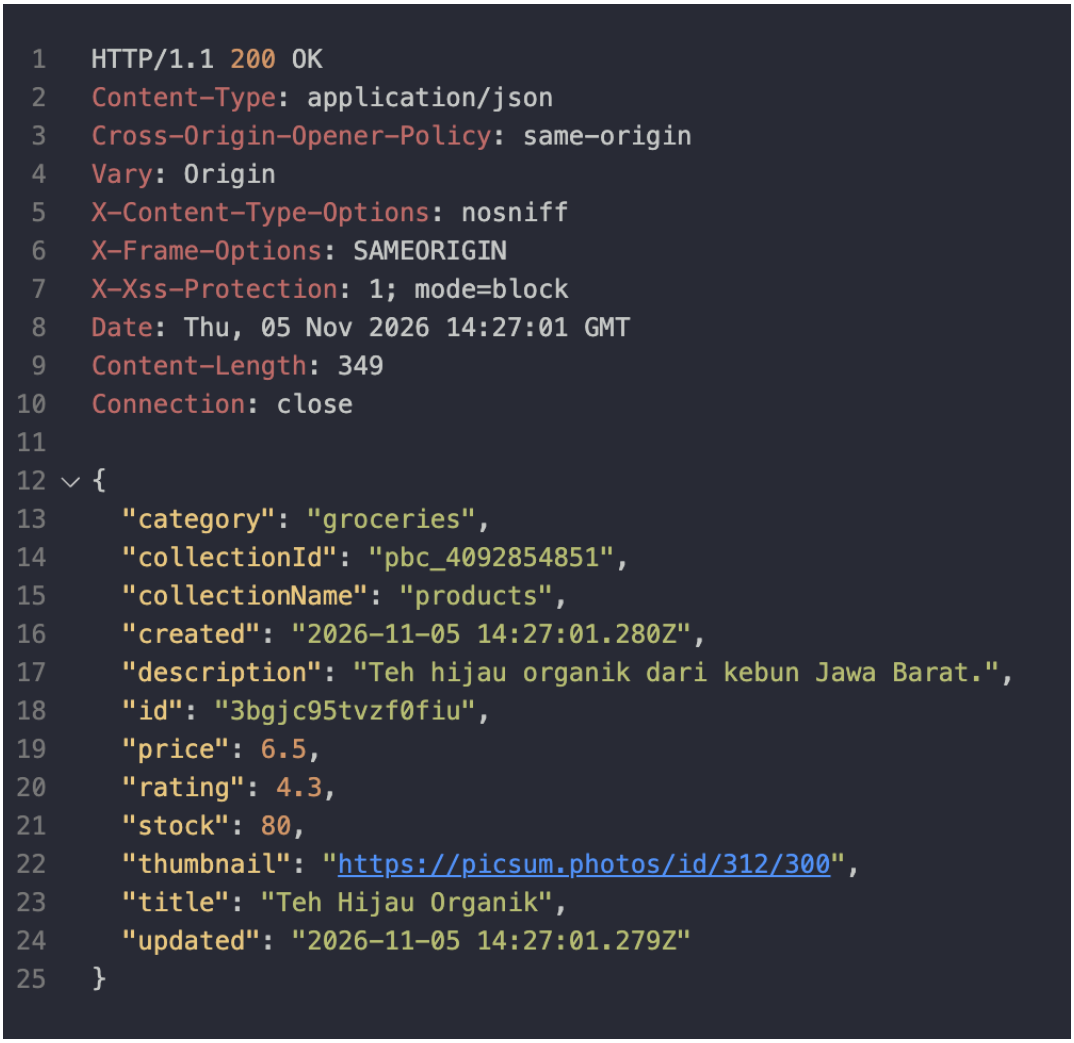
{
  "category": "fashion",
  "collectionId": "pbc_4092854851",
  "collectionName": "products",
  "created": "2026-11-05 14:23:17.292Z",
  "description": "Kemeja batik motif parang, bahan katun.",
  "id": "7eyf0zglibu9ils",
  "price": 25,
  "rating": 4.6,
  "stock": 40,
  "thumbnail": "https://picsum.photos/id/21/300",
  "title": "Kemeja Batik Parang",
  "updated": "2026-11-05 14:23:17.292Z"
}
```

Gambar 5.5 Respons penambahan produk Kemeja Batik Parang

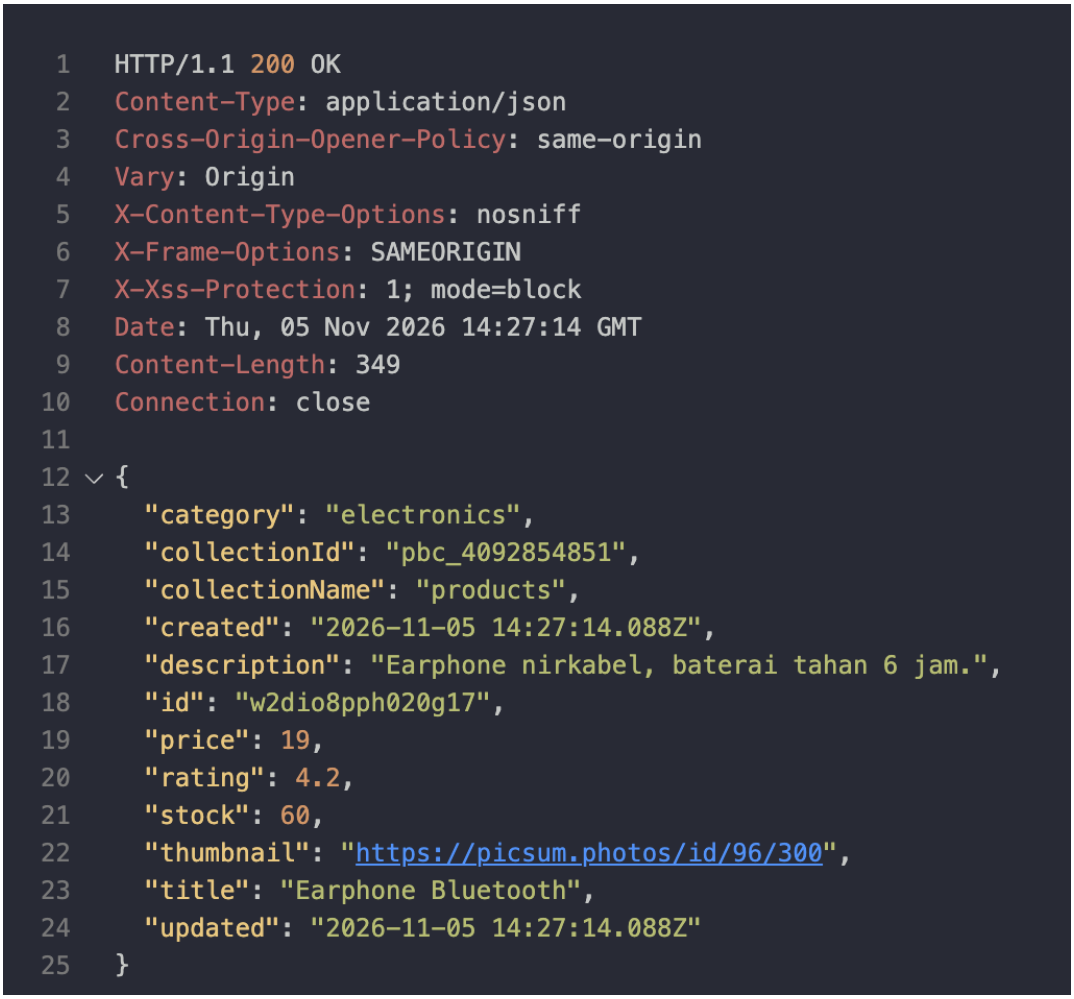
Request berikutnya dijalankan secara berurutan, yaitu Kopi Gayo 250 gr, Teh Hijau Organik, Earphone Bluetooth, dan Power Bank 10000 mAh. Seluruh request menghasilkan respons HTTP/1.1 200 OK (Gambar 5.6 sampai Gambar 5.9).



Gambar 5.6 Respons penambahan produk Kopi Gayo 250 gr



Gambar 5.7 Respons penambahan produk Teh Hijau Organik



Gambar 5.8 Respons penambahan produk Earphone Bluetooth

```
HTTP/1.1 200 OK
Content-Type: application/json
Cross-Origin-Opener-Policy: same-origin
Vary: Origin
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
X-Xss-Protection: 1; mode=block
Date: Thu, 05 Nov 2026 14:28:57 GMT
Content-Length: 347
Connection: close

{
  "category": "electronics",
  "collectionId": "pbc_4092854851",
  "collectionName": "products",
  "created": "2026-11-05 14:28:57.819Z",
  "description": "Pengisian cepat 20W, dua port USB.",
  "id": "sksyd2tmcg2iafv",
  "price": 15,
  "rating": 4.5,
  "stock": 50,
  "thumbnail": "https://picsum.photos/id/160/300",
  "title": "Power Bank 10000 mAh",
  "updated": "2026-11-05 14:28:57.819Z"
}
```


Gambar 5.9 Respons penambahan produk Power Bank 10000 mAh

Ringkasan data produk yang ditambahkan adalah sebagai berikut.

Tabel 5.1 Data produk hasil seed

Produk	Kategori	Harga	Rating	Stok
Kemeja Batik Parang	fashion	25	4.6	40
Kopi Gayo 250 gr	groceries	9	4.8	120
Teh Hijau Organik	groceries	6.5	4.3	80
Earphone Bluetooth	electronics	19	4.2	60
Power Bank 10000 mAh	electronics	15	4.5	50

BAGIAN 6: PENGUJIAN API DENGAN REST CLIENT

6.1 Penyesuaian Berkas api.http

Pengujian API melalui gateway dilakukan dengan berkas api.http. Base URL diubah agar mengarah ke gateway pada server (port 8081).

```
@baseUrl = http://100.79.171.72:8081/api
```

Variabel productId, token, dan userId terisi otomatis dari respons request sebelumnya, sehingga request dijalankan berurutan dari nomor 1 sampai 9.

6.2 Cek Gateway

```
GET {{baseUrl}}/health
```



Gambar 6.1 Request 1. Cek gateway

Respons dari server nginx/1.28.3 adalah HTTP/1.1 200 OK dengan isi {"gateway": "ok"}, yang menandakan gateway berjalan (Gambar 6.2).

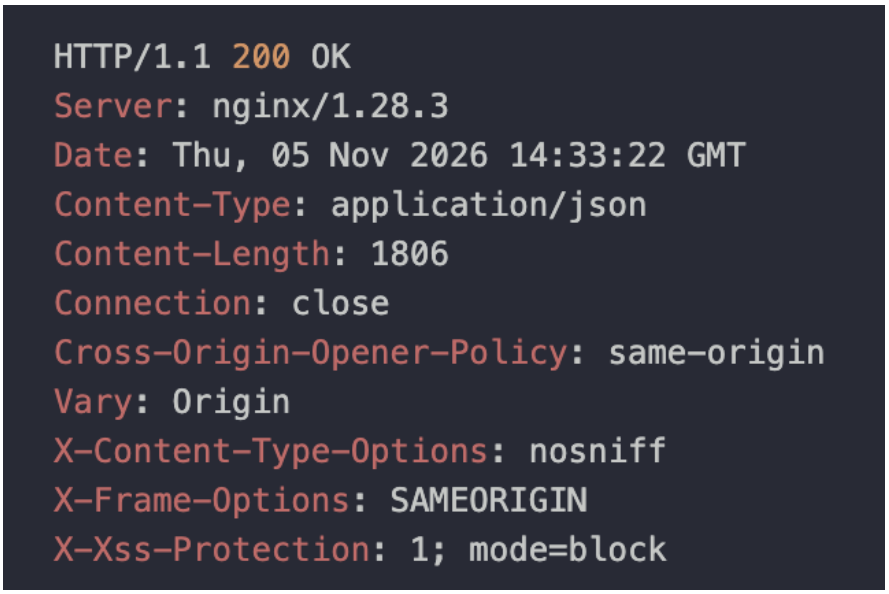


Gambar 6.2 Respons cek gateway

6.3 Daftar Produk

```
GET {{baseUrl}}/products?perPage=5&sort=-created
```

Gateway meneruskan request ke product-service dan mengembalikan 200 OK berisi lima produk terbaru (Gambar 6.3).

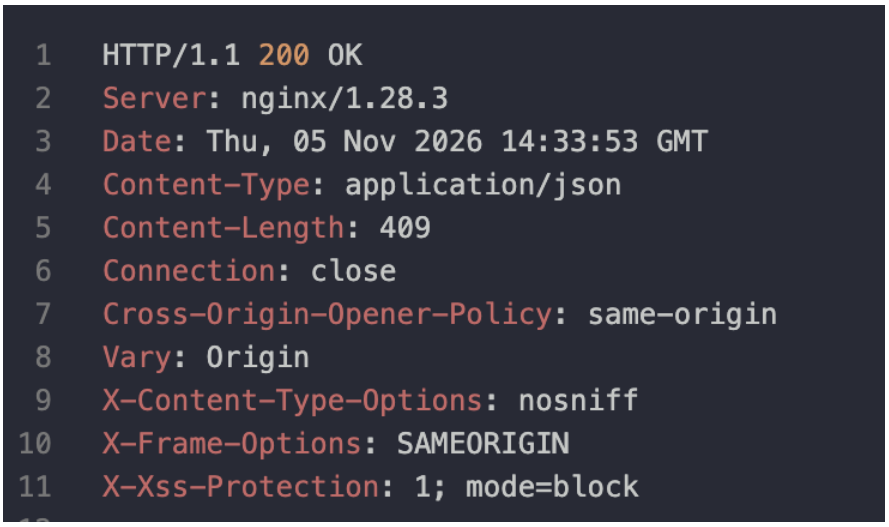


Gambar 6.3 Respons daftar produk (5 terbaru)

6.4 Pencarian Produk dengan Filter

```
GET {{baseUrl}}/products?filter=title~'kopi'
```

Filter PocketBase title~'kopi' menghasilkan respons 200 OK berisi produk yang judulnya mengandung kata kopi (Gambar 6.4).

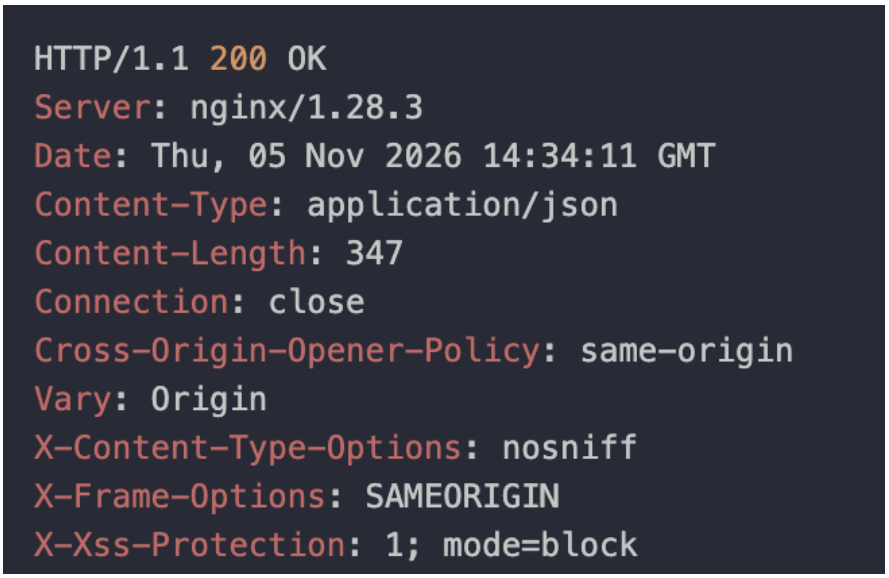


Gambar 6.4 Respons pencarian produk dengan filter

6.5 Detail Produk

```
GET {{baseUrl}}/products/{{productId}}
```

Detail produk berdasarkan productId hasil request daftar produk berhasil diambil dengan status 200 OK (Gambar 6.5).



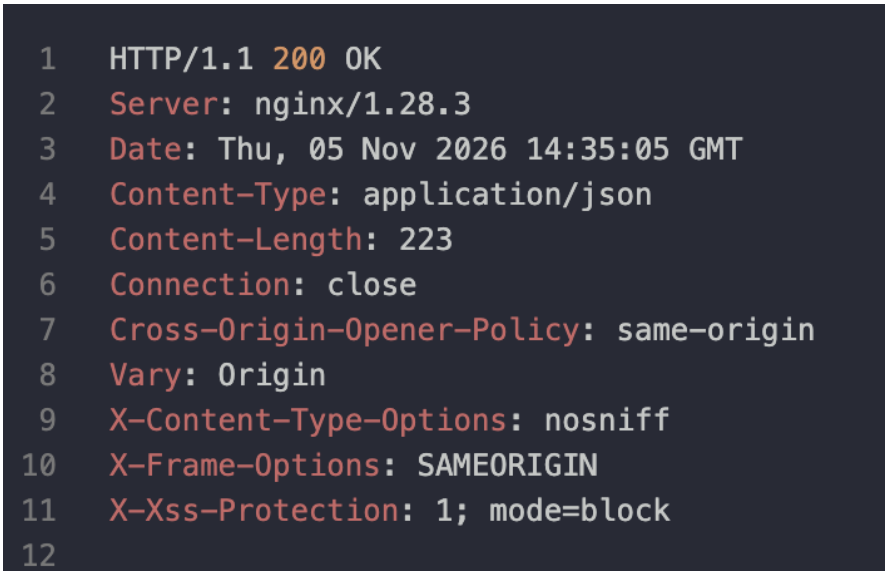
Gambar 6.5 Respons detail produk

6.6 Pendaftaran Akun

```
POST {{baseUrl}}/auth/records
Content-Type: application/json

{
 "name": "Budi",
 "email": "budi@itb.ac.id",
 "password": "*******",
 "passwordConfirm": "*******"
}
```

Request diteruskan gateway ke koleksi users pada account-service dan menghasilkan 200 OK, artinya akun baru berhasil dibuat (Gambar 6.6).



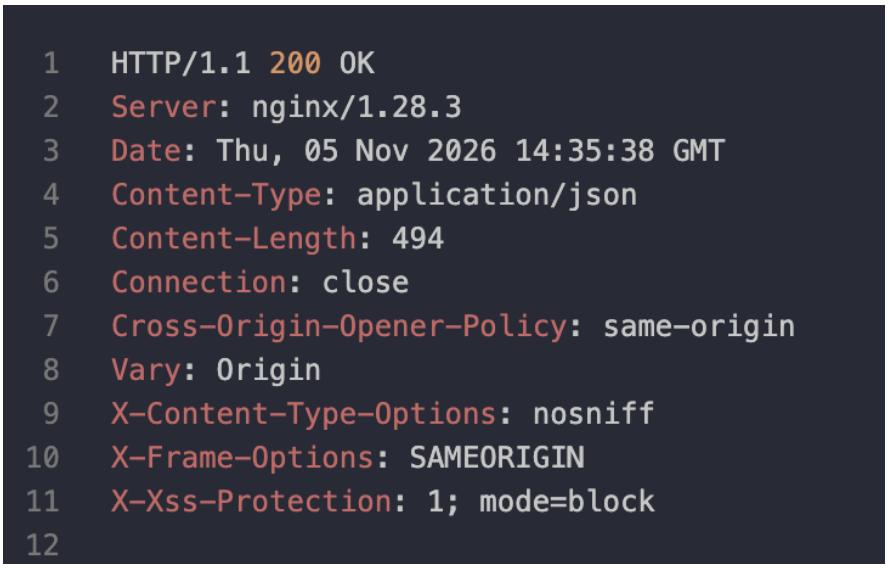
Gambar 6.6 Respons pendaftaran akun

6.7 Login

```
# @name login
POST {{baseUrl}}/auth/auth-with-password
Content-Type: application/json

{
  "identity": "budi@itb.ac.id",
  "password": "*******"
}
```

Login berhasil dengan status 200 OK (Gambar 6.7). Respons berisi token dan record user yang disimpan otomatis ke variabel token dan userId.



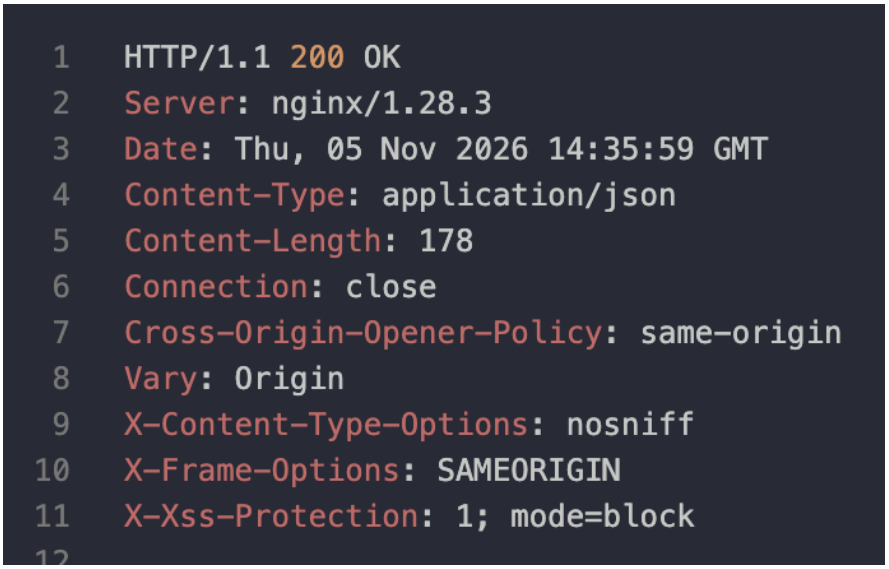
Gambar 6.7 Respons login

6.8 Tambah Favorit

```
POST {{baseUrl}}/favorites
Authorization: Bearer {{token}}
Content-Type: application/json

{
  "user": "{{userId}}",
  "product_id": "{{productId}}"
}
```

Dengan token login, produk berhasil ditambahkan ke favorit milik user (200 OK, Gambar 6.8).

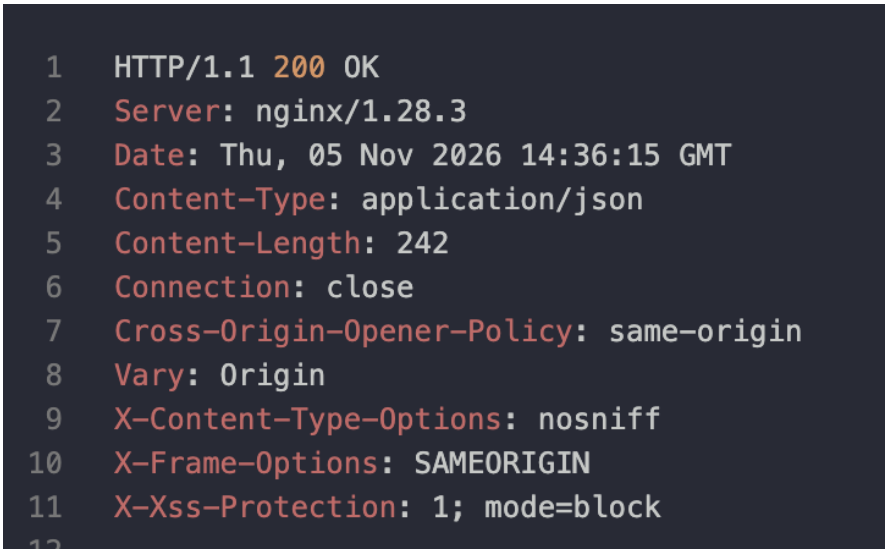


Gambar 6.8 Respons tambah favorit

6.9 Lihat Favorit Saya

```
# @name favs
GET {{baseUrl}}/favorites
Authorization: Bearer {{token}}
```

Respons 200 OK pada Gambar 6.9 berisi daftar favorit milik user yang sedang login.

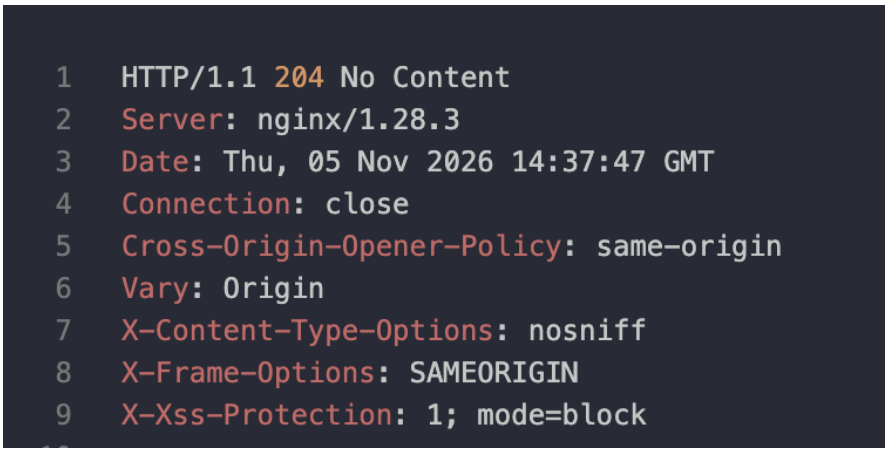


Gambar 6.9 Respons lihat favorit

6.10 Hapus Favorit

```
DELETE {{baseUrl}}/favorites/{{favs.response.body.$.items[0].id}}
Authorization: Bearer {{token}}
```

Favorit pertama berhasil dihapus dengan respons HTTP/1.1 204 No Content (Gambar 6.10).



Gambar 6.10 Respons hapus favorit

6.11 Ringkasan Pengujian REST Client

Tabel 6.1 Ringkasan hasil pengujian API dengan REST Client

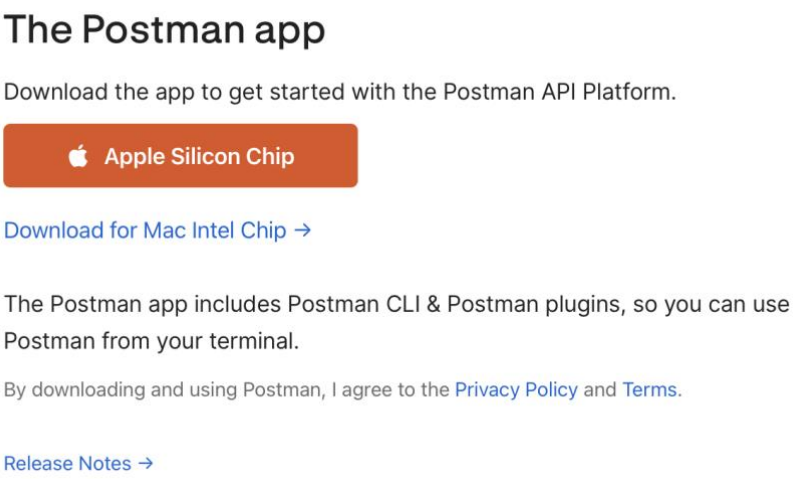
No	Method	Endpoint	Hasil
1	GET	/api/health	200 OK, {"gateway":"ok"}
2	GET	/api/products?perPage=5&sort=-created	200 OK, daftar produk
3	GET	/api/products?filter=title~'kopi'	200 OK, hasil pencarian
4	GET	/api/products/:id	200 OK, detail produk
5	POST	/api/auth/records	200 OK, akun baru

No	Method	Endpoint	Hasil
6	POST	/api/auth/auth-with-password	200 OK, token dan record
7	POST	/api/favorites	200 OK, favorit ditambahkan
8	GET	/api/favorites	200 OK, favorit milik sendiri
9	DELETE	/api/favorites/:id	204 No Content

BAGIAN 7: PENGUJIAN API DENGAN POSTMAN

7.1 Instalasi Postman

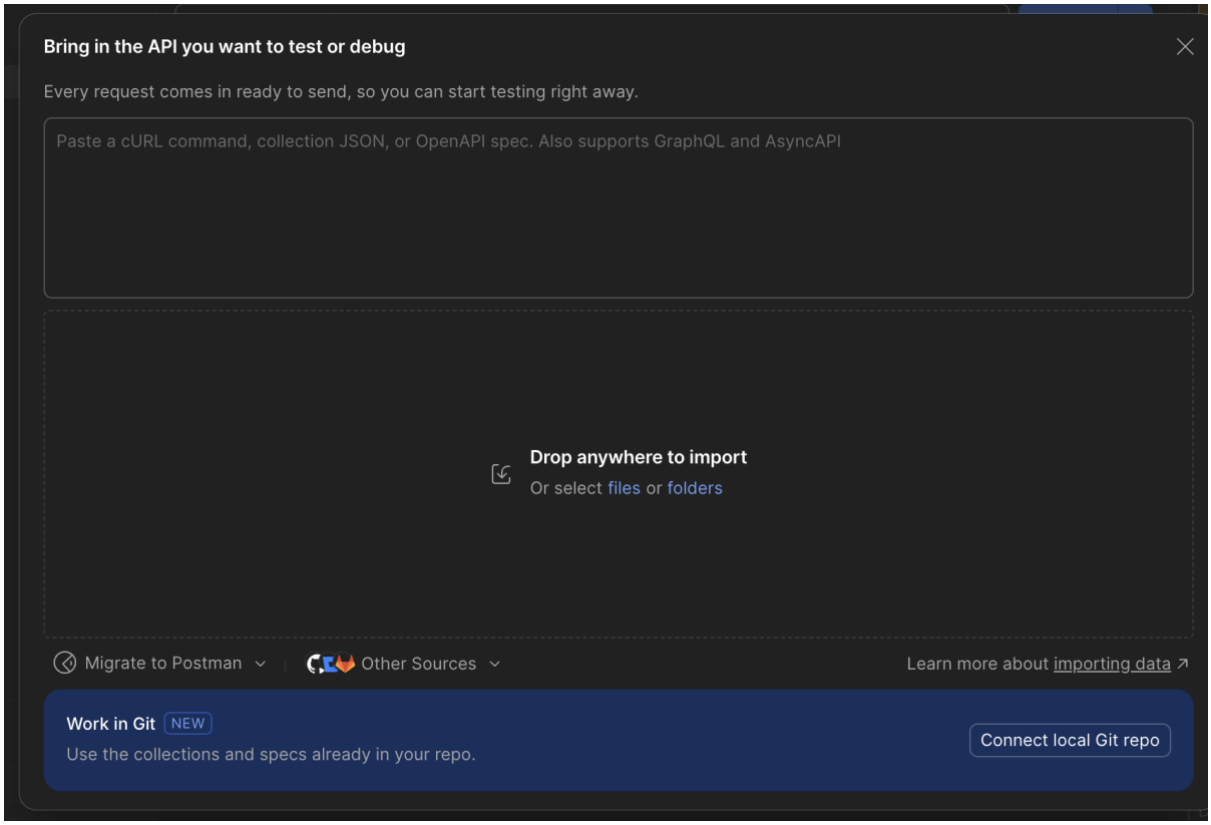
Postman Desktop diunduh dari https://www.postman.com/downloads/ (versi Apple Silicon Chip), lalu dipasang pada MacBook.



Gambar 7.1 Halaman unduhan Postman

7.2 Import Koleksi dan Environment

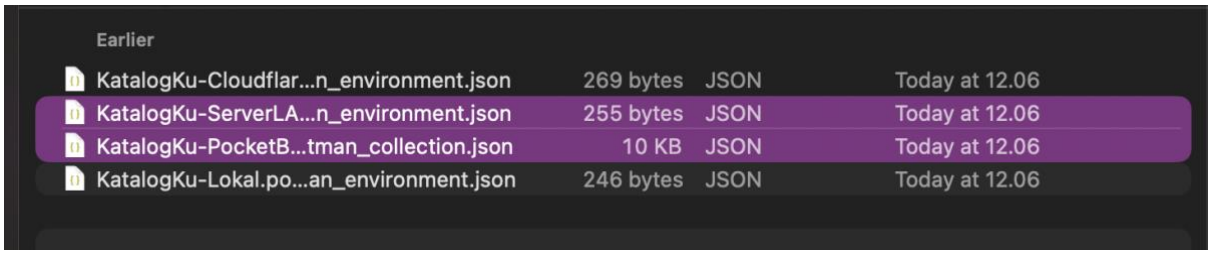
Pada Postman ditekan tombol Import, dipilih tab Files, lalu Choose Files.



Gambar 7.2 Jendela Import pada Postman

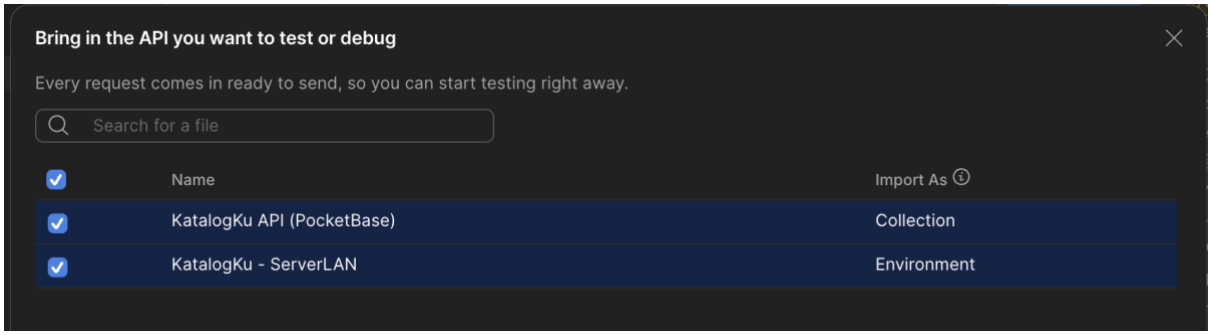
Dari folder Downloads/katalogku-pocketbase/postman/ dipilih dua berkas berikut.

1. KatalogKu-PocketBase.postman_collection.json
2. KatalogKu-ServerLAN.postman_environment.json



Gambar 7.3 Pemilihan berkas koleksi dan environment Postman

Gambar 7.4 menunjukkan KatalogKu API (PocketBase) akan diimpor sebagai Collection dan KatalogKu - ServerLAN sebagai Environment.

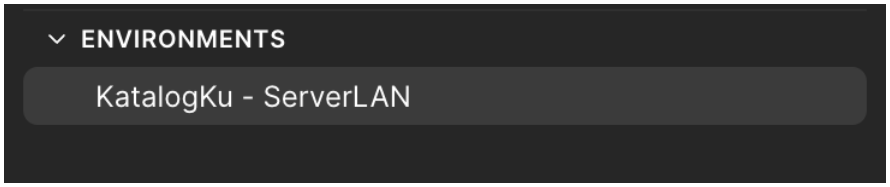


Gambar 7.4 Konfirmasi import koleksi dan environment

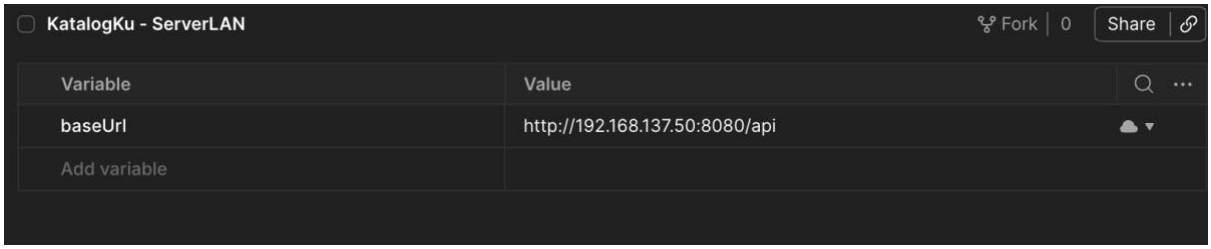
7.3 Pengaturan Environment ServerLAN

Environment KatalogKu - ServerLAN dibuka dari menu Environments. Nilai awal variabel baseUrl adalah http://192.168.137.50:8080/api (Gambar 7.6), sehingga diganti menjadi alamat gateway pada server, lalu environment diatur sebagai aktif (set active).

baseUrl = http://100.79.171.72:8081/api



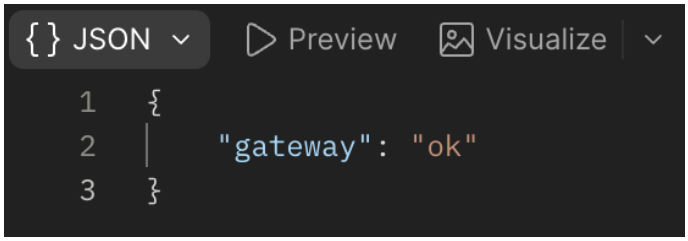
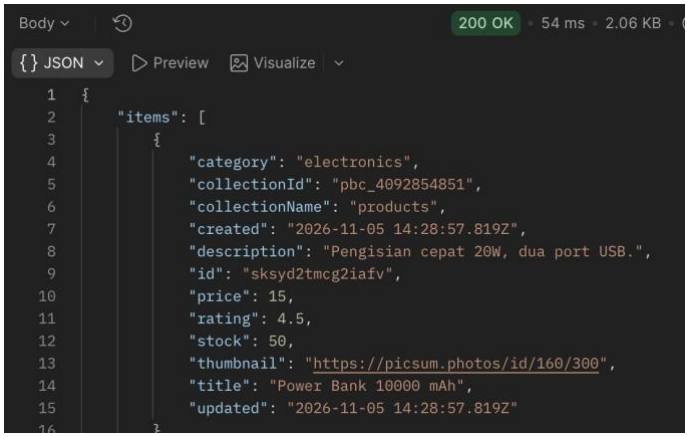
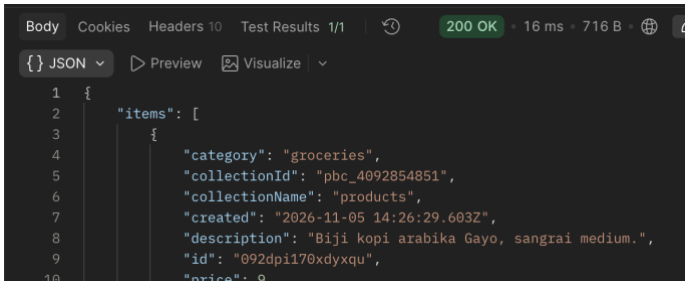
Gambar 7.5 Environment KatalogKu - ServerLAN



Gambar 7.6 Variabel baseUrl sebelum diubah

7.4 Hasil Pengujian Postman

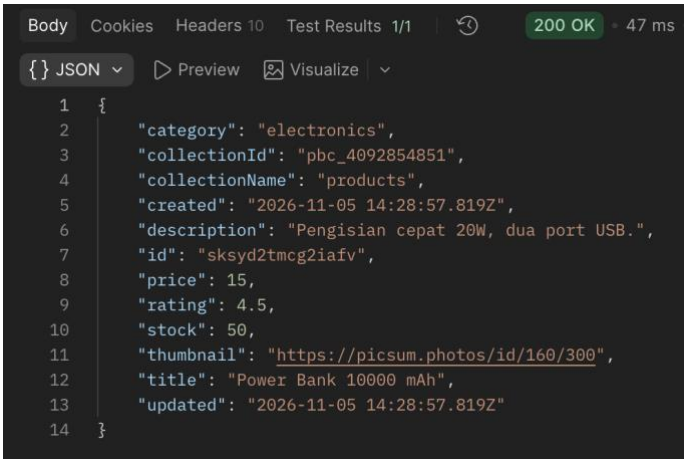
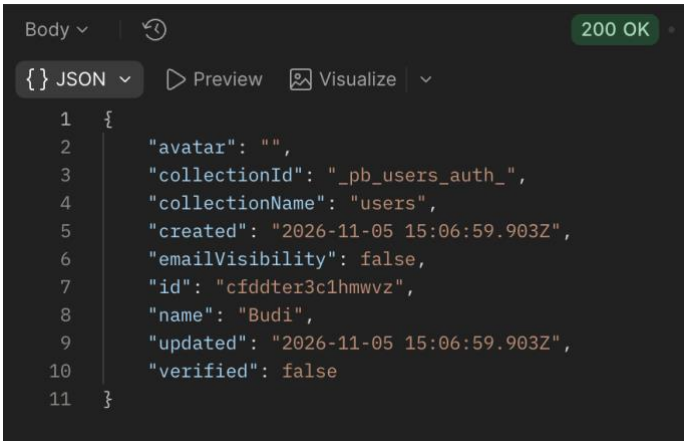
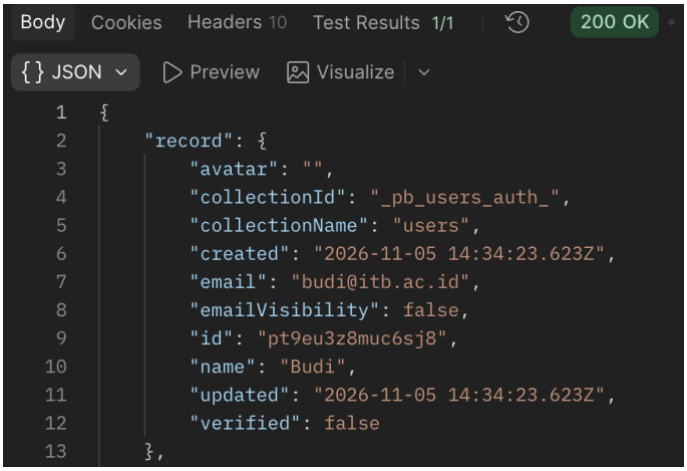
Request pada koleksi KatalogKu-PocketBase dijalankan secara berurutan. Script pada koleksi menyimpan productId, token, dan userId secara otomatis. Hasil setiap request adalah sebagai berikut.

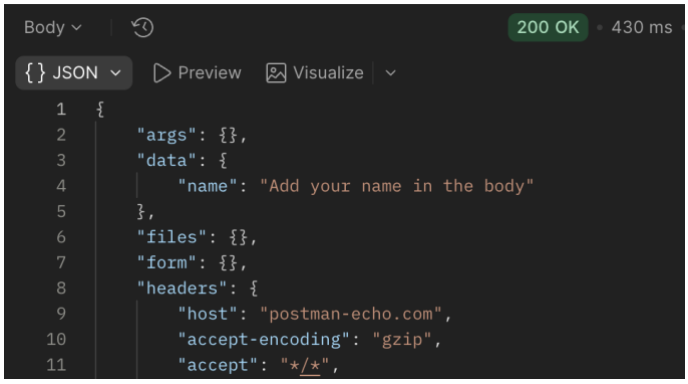
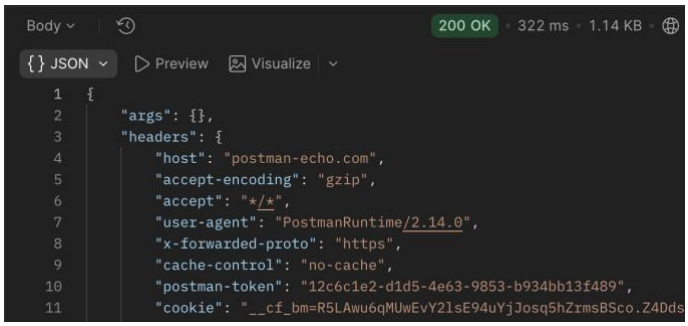
No	Method	Nama request	Hasil
1	GET	Cek gateway	
2	GET	Daftar produk	
3	GET	Cari produk	

Gambar 7.7 Postman: cek gateway

Gambar 7.8 Postman: daftar produk

Gambar 7.9 Postman: cari produk

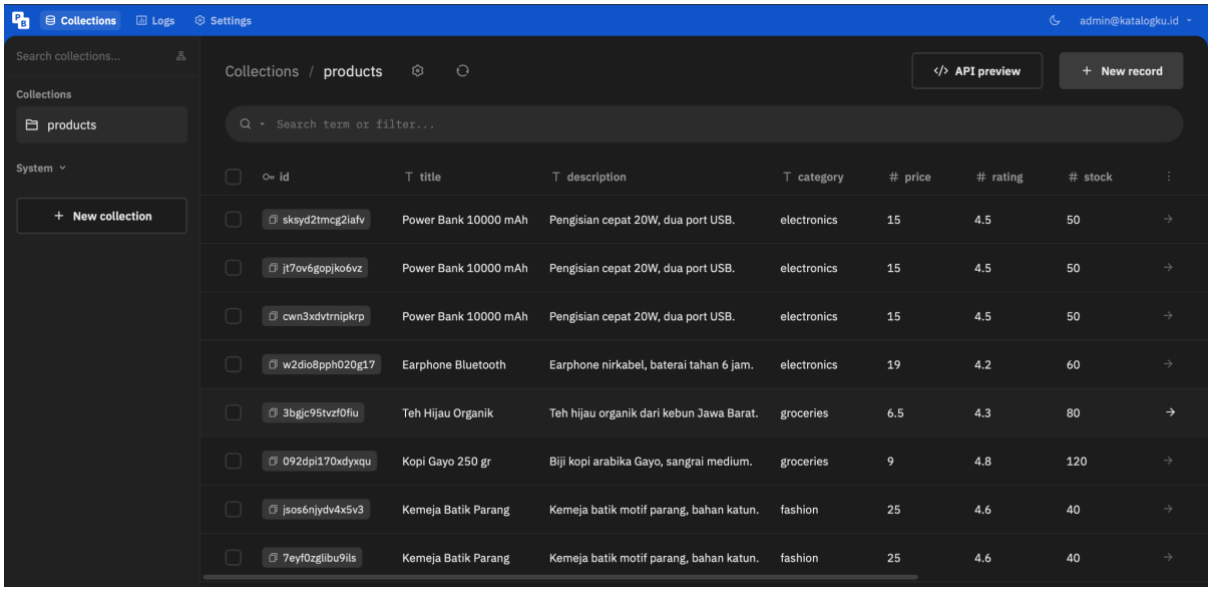
No	Method	Nama request	Hasil
4	GET	Detail produk	 *Gambar 7.10 Postman: detail produk*
5	POST	Daftar akun	 *Gambar 7.11 Postman: daftar akun*
6	POST	Login (simpan token)	 *Gambar 7.12 Postman: login*

No	Method	Nama request	Hasil
7	POST	Tambah favorit	 *Gambar 7.13 Postman: tambah favorit*
8	GET	Lihat favorit	 *Gambar 7.14 Postman: lihat favorit*

BAGIAN 8: BACKUP DATA DAN UJI REBOOT

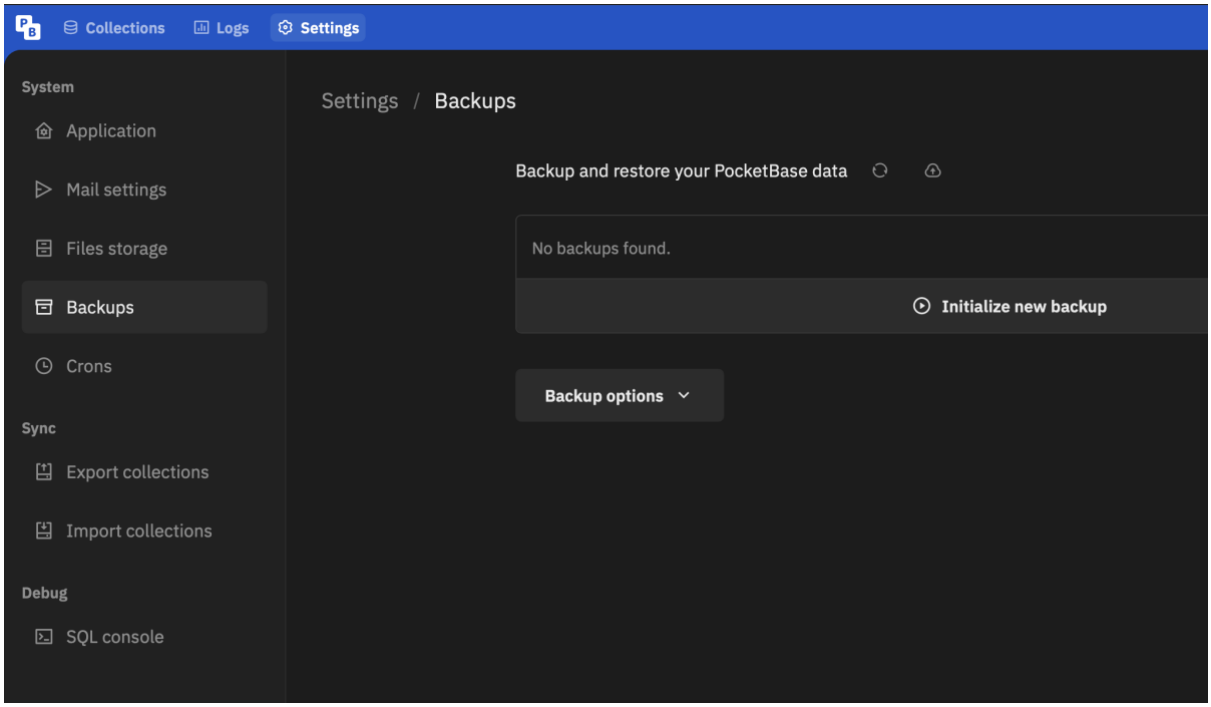
8.1 Backup Product Service

Dashboard product-service dibuka pada http://100.79.171.72:8091/_/. Koleksi products sudah berisi data hasil seed (Gambar 8.1). Beberapa produk tercatat lebih dari sekali karena request seed dikirim lebih dari satu kali.



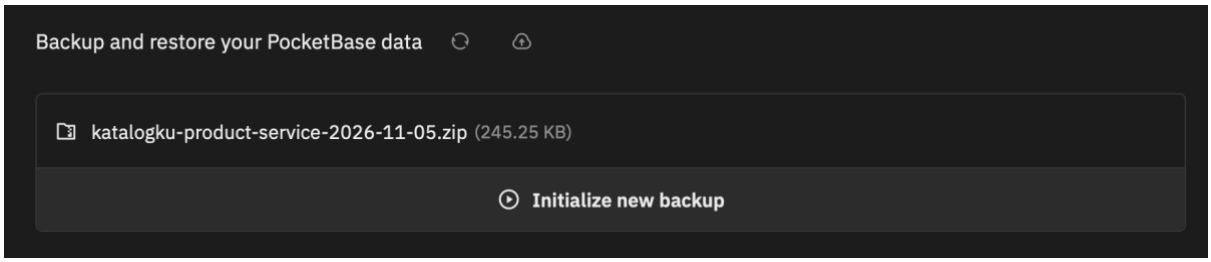
Gambar 8.1 Isi koleksi products pada dashboard product-service

Backup dibuat melalui menu Settings, Backups, lalu Initialize new backup.



Gambar 8.2 Halaman Backups pada dashboard product-service

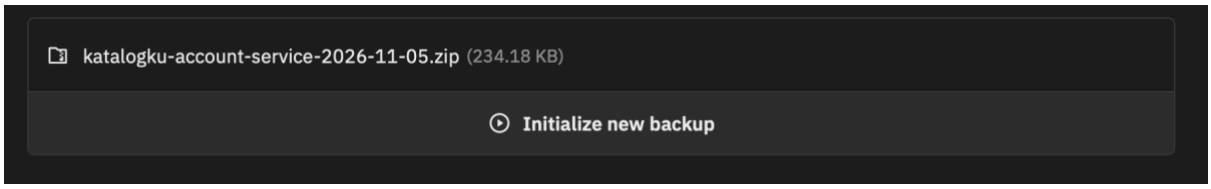
Backup diberi nama katalogku-product-service-2026-11-05.zip dan berhasil dibuat dengan ukuran 245,25 KB (Gambar 8.3).



Gambar 8.3 Backup product-service berhasil dibuat

8.2 Backup Account Service

Langkah yang sama dilakukan pada dashboard account-service (http://100.79.171.72:8092/_/). Backup diberi nama katalogku-account-service-2026-11-05.zip dan berhasil dibuat dengan ukuran 234,18 KB.



Gambar 8.4 Backup account-service berhasil dibuat

8.3 Pemeriksaan Container dan Restart Policy

Melalui terminal SSH di VS Code, container yang berjalan diperiksa kembali.

```
docker ps
```

Gambar 8.5 menunjukkan lima container berjalan, yaitu gateway, account-service, product-service, phpmyadmin, dan mysql-server.

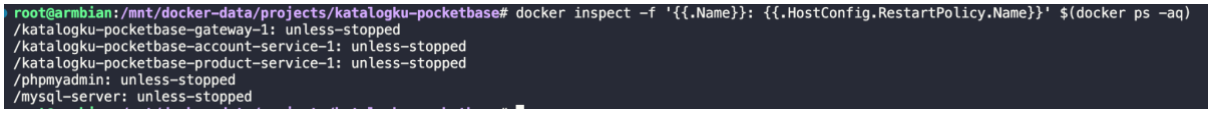


Gambar 8.5 Daftar container yang berjalan

Restart policy setiap container diperiksa dengan perintah berikut.

```
docker inspect -f '{{.Name}}: {{.HostConfig.RestartPolicy.Name}}' $(docker ps -aq)
```

Seluruh container menggunakan restart policy unless-stopped (Gambar 8.6), sehingga container akan dijalankan kembali secara otomatis ketika Docker aktif setelah server dinyalakan ulang.



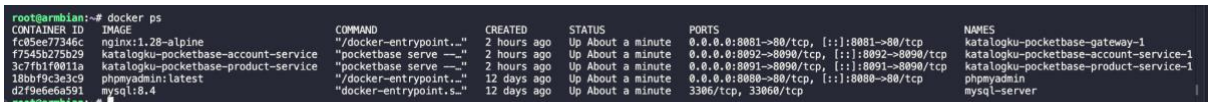
Gambar 8.6 Restart policy seluruh container

8.4 Bukti Hasil: Uji Reboot STB

STB kemudian di-reboot untuk menguji ketahanan layanan. Setelah server menyala kembali, docker ps dijalankan dari SSH.

```
reboot
docker ps
```

Pengujian berhasil. Kelima container otomatis aktif kembali setelah STB reboot dengan status Up About a minute, tanpa perlu menjalankan docker compose up secara manual (Gambar 8.7).



Gambar 8.7 Container aktif kembali setelah STB reboot